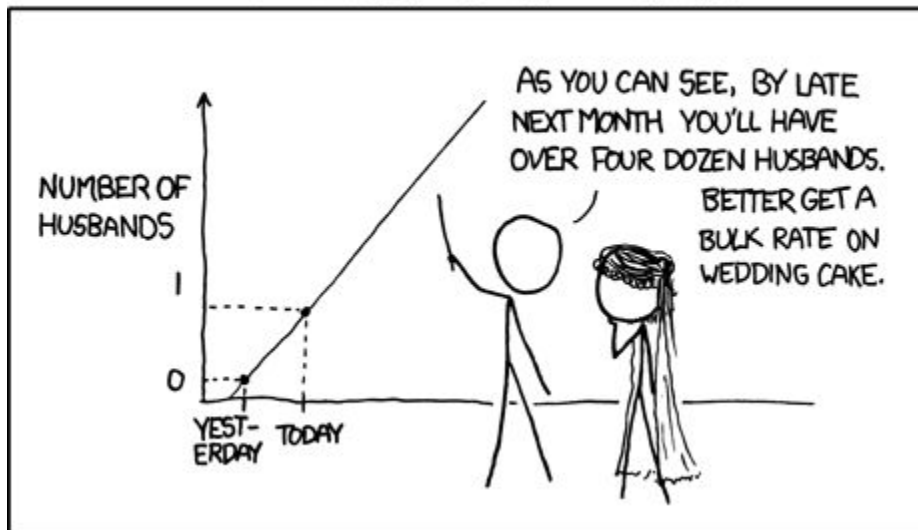

Series Temporales: Forecasting parte 2

Federico Albanese

(falbanese@dc.uba.ar)

 @f__albanese

MY HOBBY: EXTRAPOLATING



¿ Qué vamos a ver hoy?

¿ Qué vamos a ver hoy?

- Deep Learning:
 - Neural Networks
 - CNN: Convolutional Neural Network (LeCun, 1989)
 - RNN: Recurrent Neural Network (Rumelhart et al., 1986a)
 - LSTM: Long Short-Term Memory (Hochreiter and Schmidhuber, 1997).

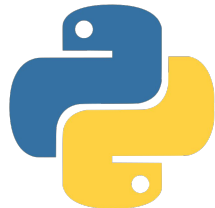
¿ Qué vamos a ver hoy?

- Deep Learning:
 - Neural Networks
 - CNN: Convolutional Neural Network (LeCun, 1989)
 - RNN: Recurrent Neural Network (Rumelhart et al., 1986a)
 - LSTM: Long Short-Term Memory (Hochreiter and Schmidhuber, 1997).
- Prophet (Taylor et al., 2018)

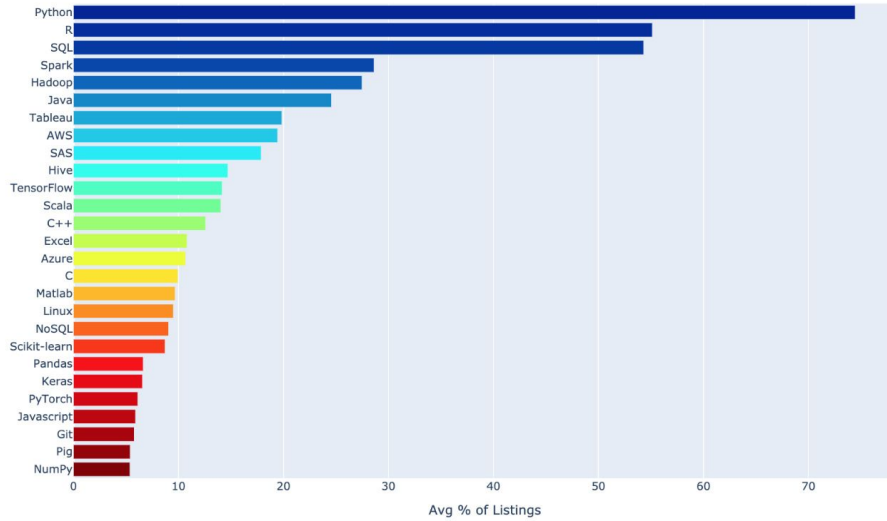
¿ Qué vamos a ver hoy?

- Deep Learning:
 - Neural Networks
 - CNN: Convolutional Neural Network (LeCun, 1989)
 - RNN: Recurrent Neural Network (Rumelhart et al., 1986a)
 - LSTM: Long Short-Term Memory (Hochreiter and Schmidhuber, 1997).
- Prophet (Taylor et al., 2018)

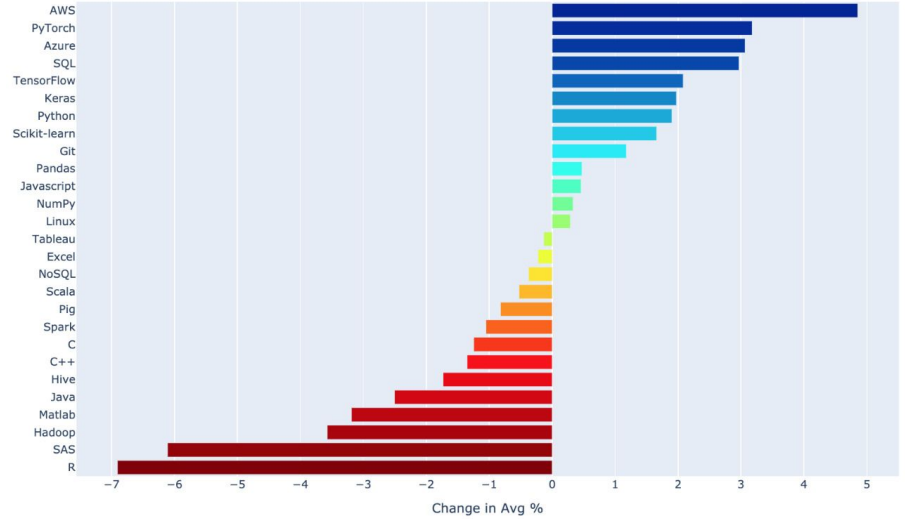
¿ Qué herramientas vamos a usar?



Technologies in Data Scientist Job Listings 2019



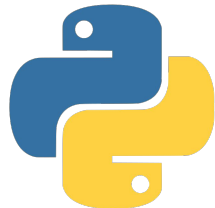
Change in Avg % of Technologies in Data Scientist Job Listings 2019



¿ Qué vamos a ver hoy?

- Deep Learning:
 - Neural Networks
 - CNN: Convolutional Neural Network (LeCun, 1989)
 - RNN: Recurrent Neural Network (Rumelhart et al., 1986a)
 - LSTM: Long Short-Term Memory (Hochreiter and Schmidhuber, 1997).
- Prophet (Taylor et al., 2018)

¿ Qué herramientas vamos a usar?



Does everyone understand?



(silent)



**Good, I'll move
on then**



ONE DOES NOT SIMPLY

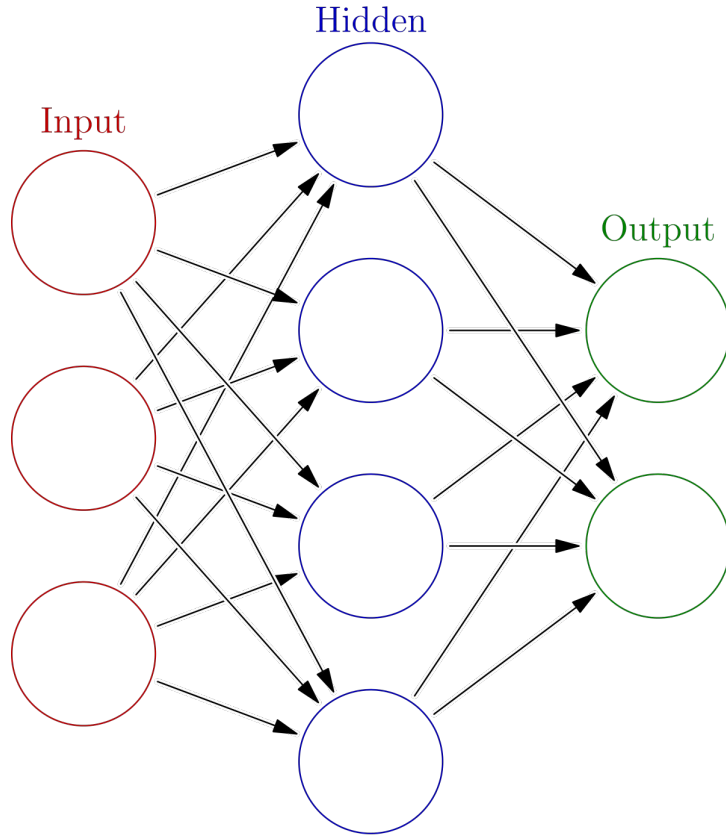
USE MEMES IN THE CLASSROOM



Neural Networks

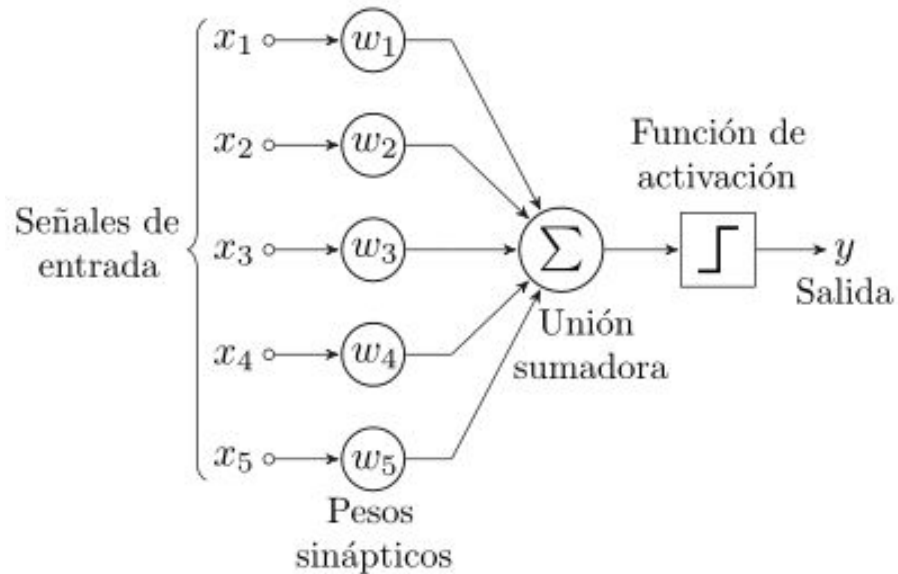


Red Neuronal

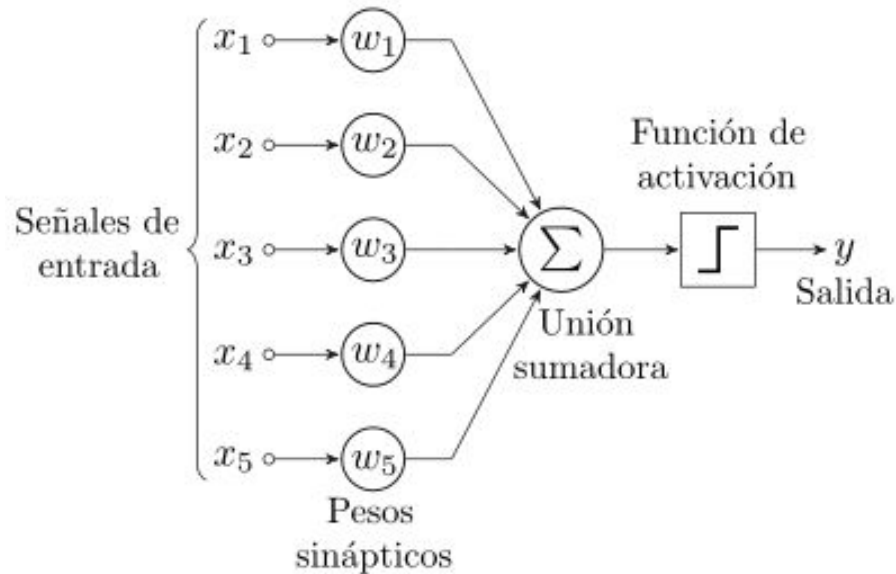


“Las **redes neuronales** son un modelo computacional. La información de entrada atraviesa la red neuronal (donde se somete a diversas operaciones) produciendo unos valores de salida.” (wikipedia)

Red Neuronal: perceptrón simple

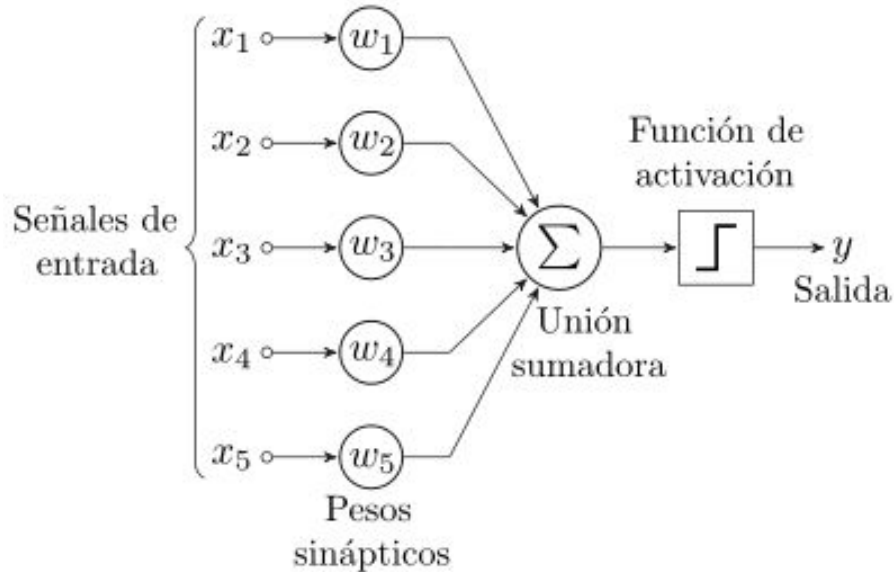


Red Neuronal: perceptrón simple



$$Y = \sigma(\sum W_i * X_i + b)$$

Red Neuronal: perceptrón simple

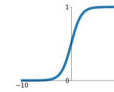


$$Y = \sigma(\Sigma W_i * X_i + b)$$

Activation Functions

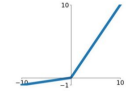
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



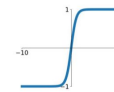
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

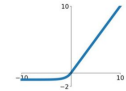


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

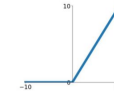
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

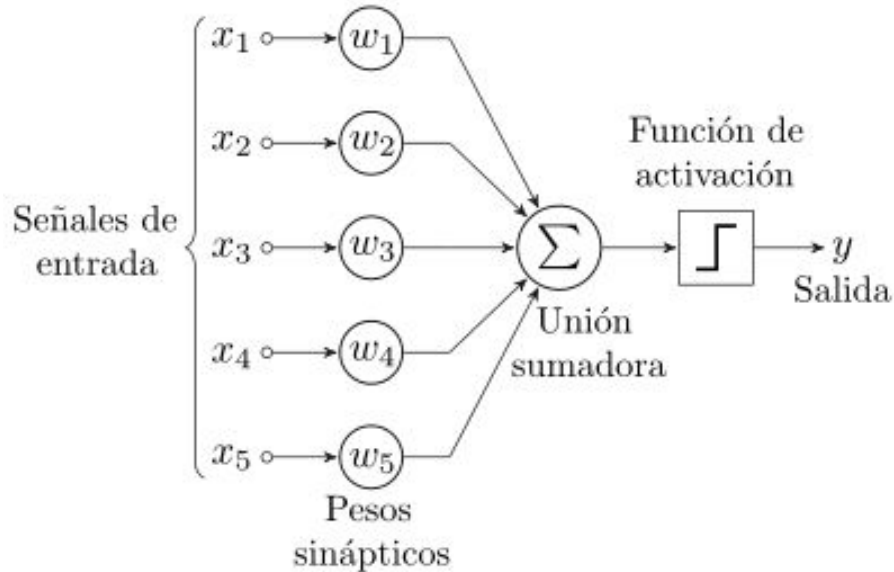


ReLU

$$\max(0, x)$$



Red Neuronal: perceptrón simple

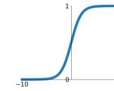


$$Y = \sigma(\Sigma W_i * X_i + b)$$

Activation Functions

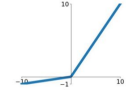
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



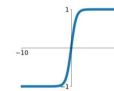
Leaky ReLU

$$\max(0.1x, x)$$



tanh

$$\tanh(x)$$

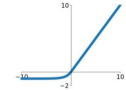


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

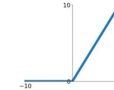
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



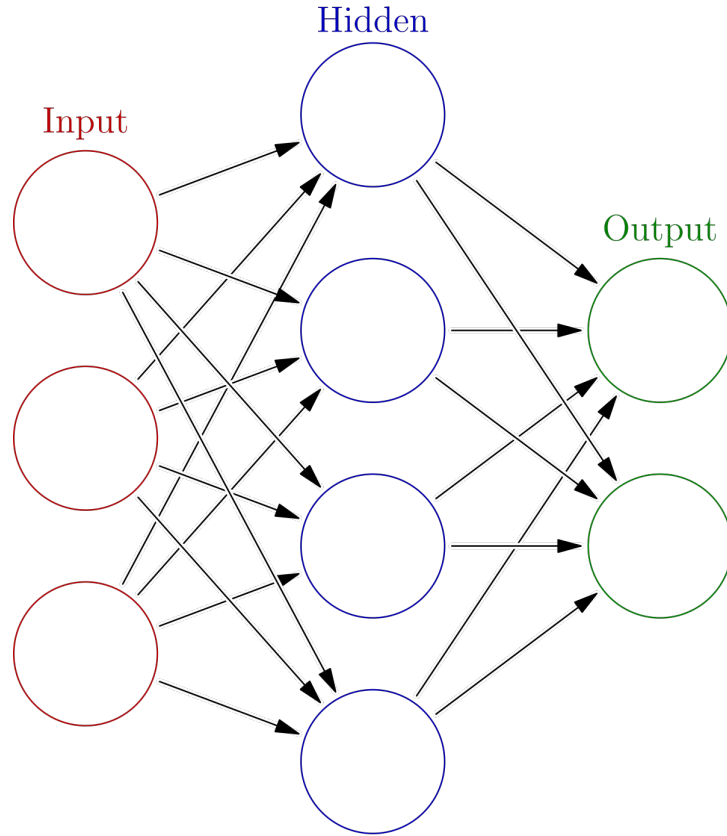
ReLU

$$\max(0, x)$$



¿Cuánto valen los pesos y los bias?

Red Neuronal



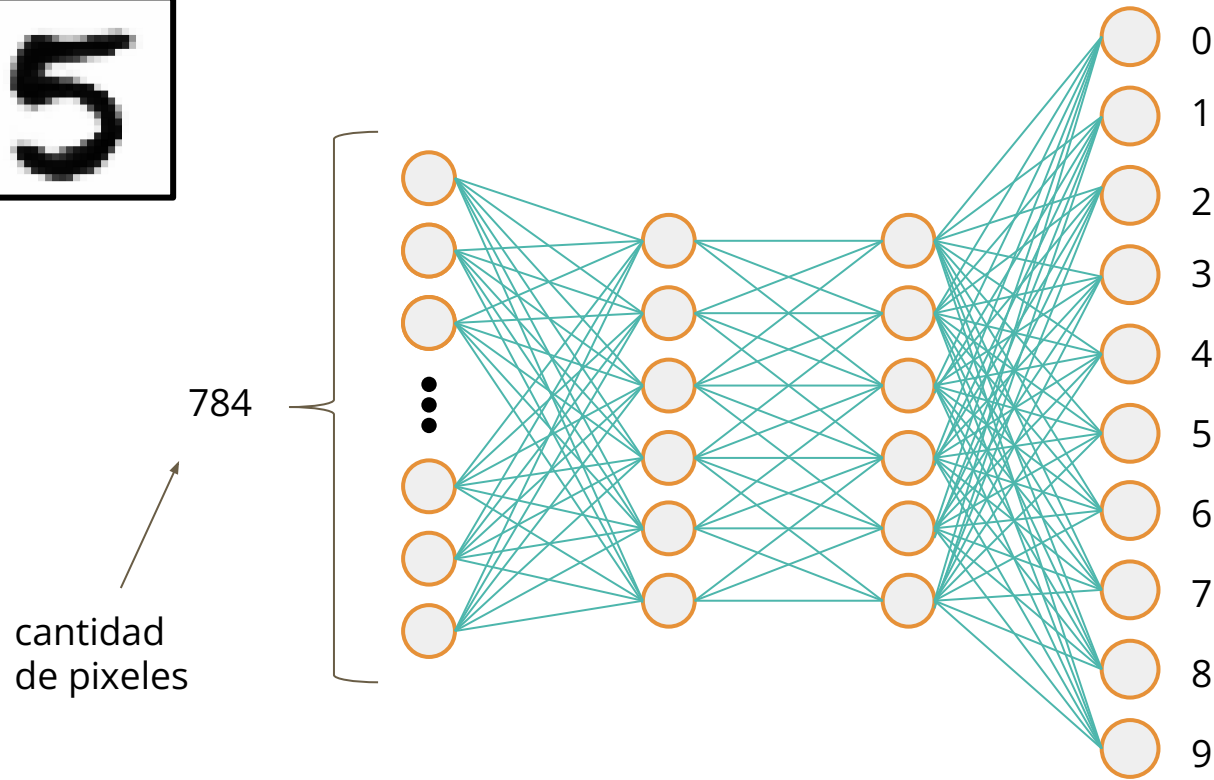
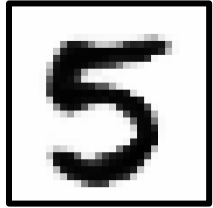
Una Red Neuronal solo son muchos perceptrones simples conectados entre sí.

Red Neuronal

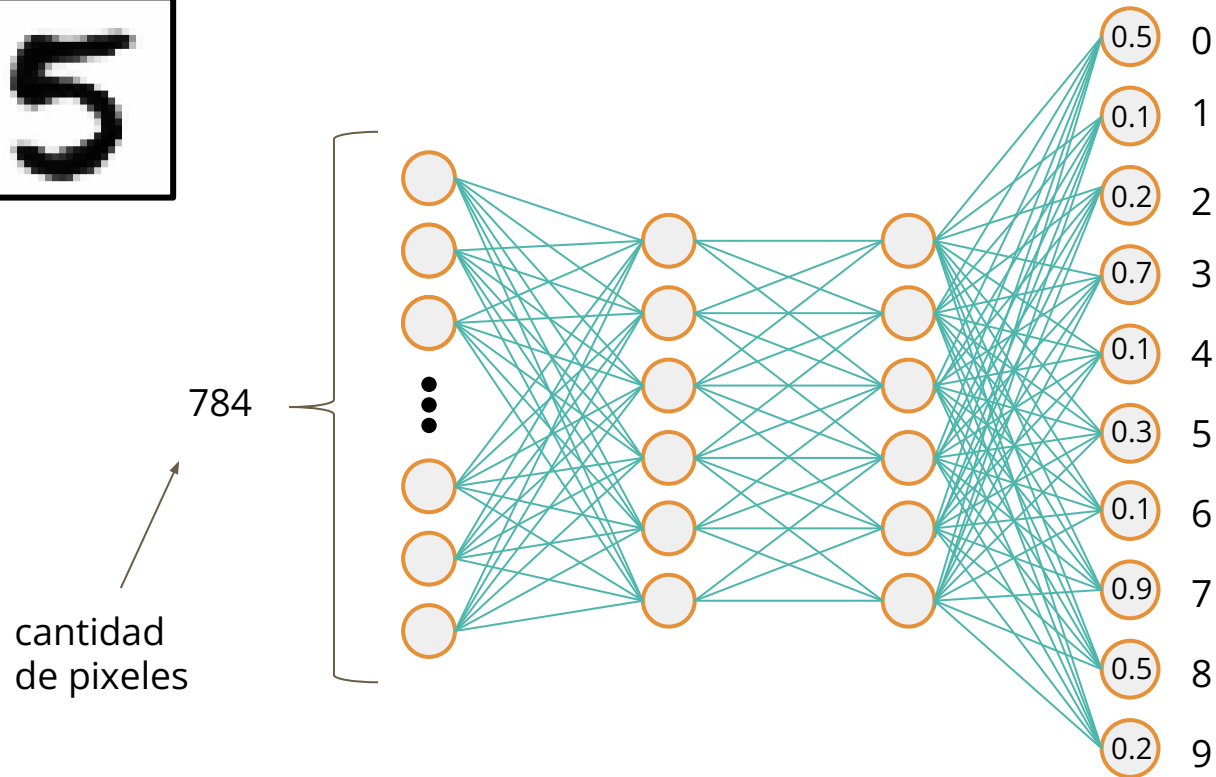
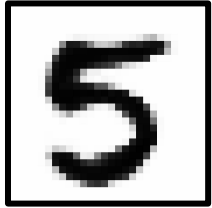
Ejemplo: mnist



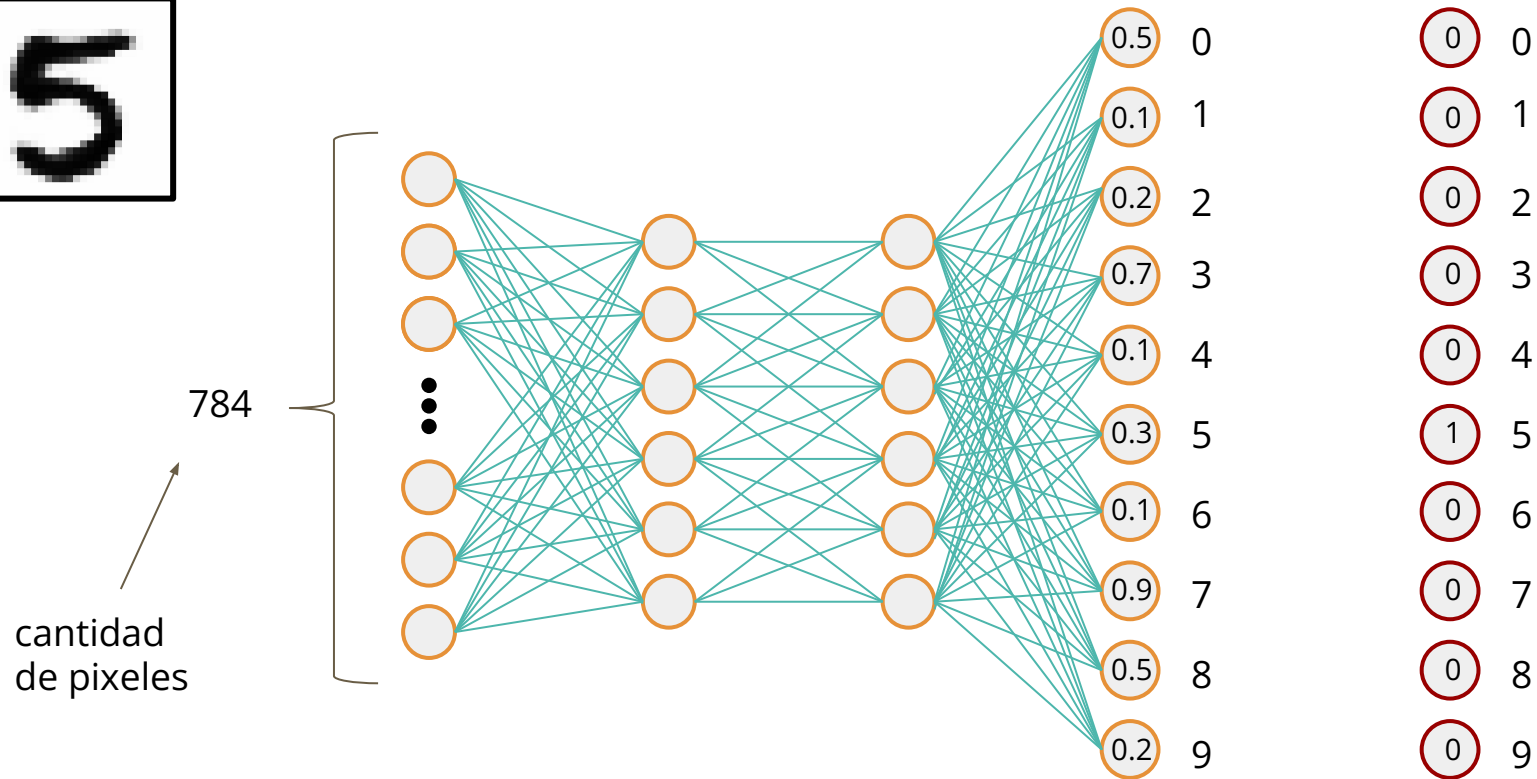
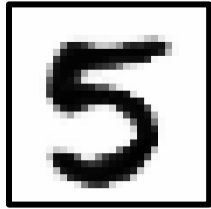
Red Neuronal: Backpropagation o cómo calcular w y b



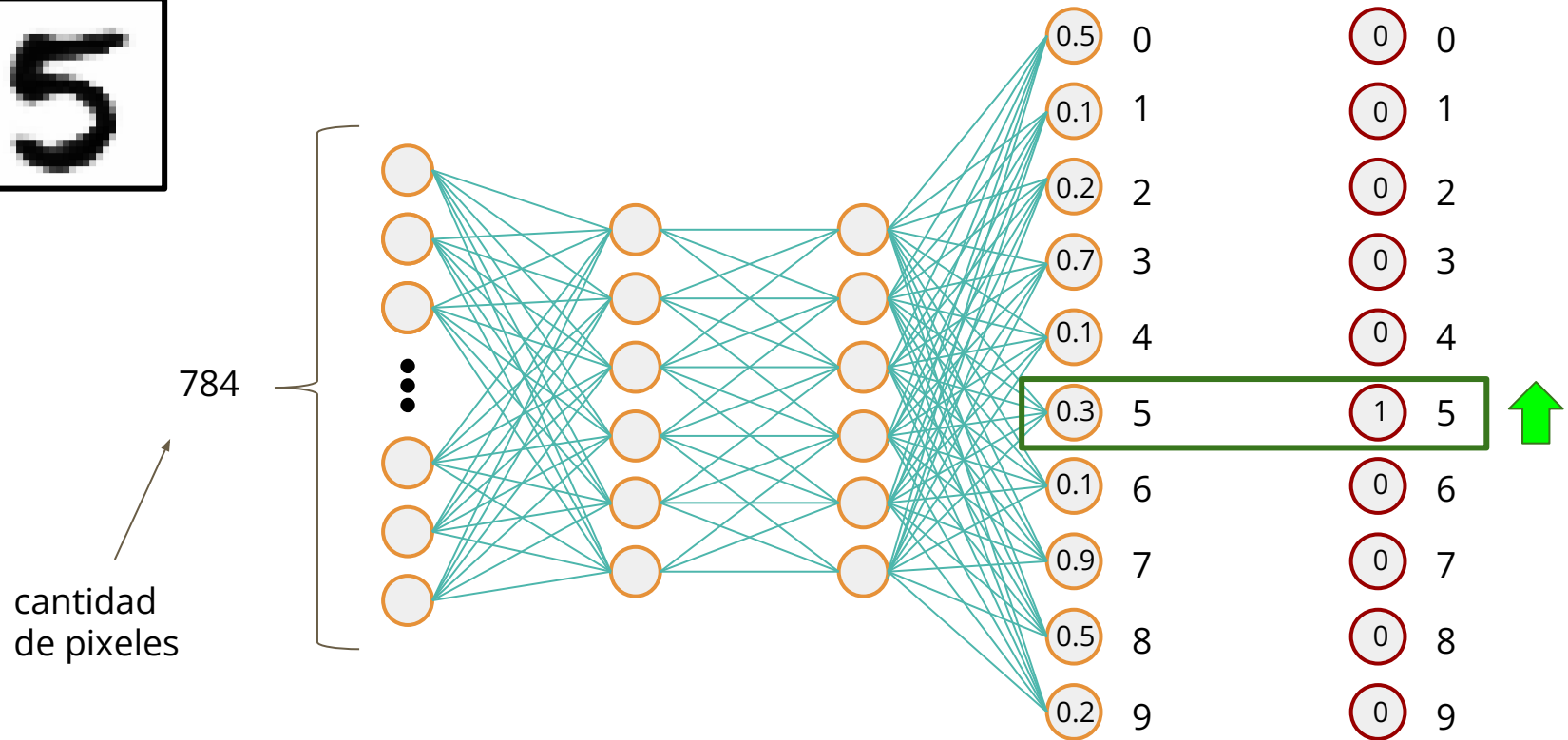
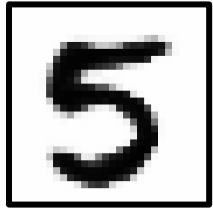
Red Neuronal: Backpropagation o cómo calcular w y b



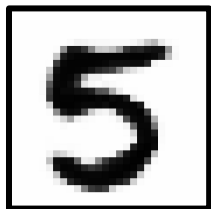
Red Neuronal: Backpropagation o cómo calcular w y b



Red Neuronal: Backpropagation o cómo calcular w y b

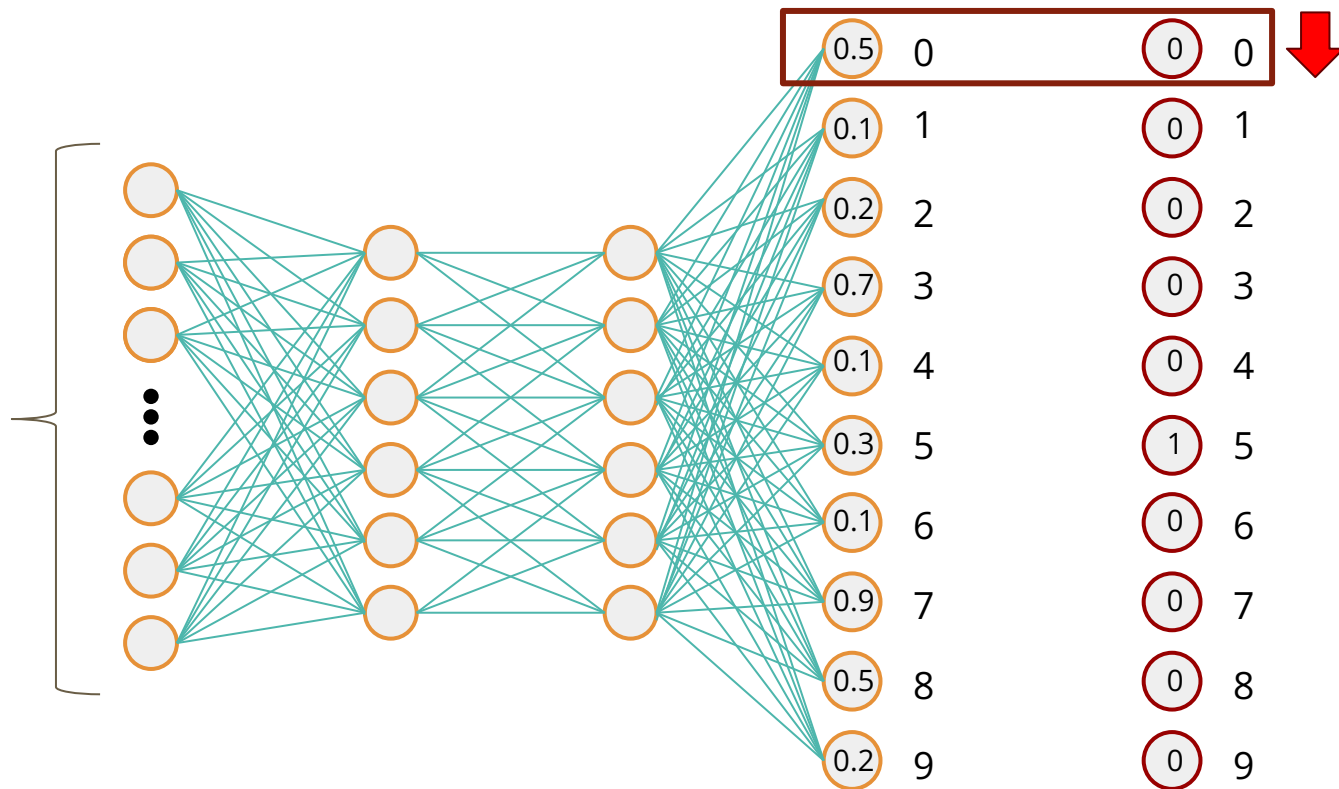


Red Neuronal: Backpropagation o cómo calcular w y b

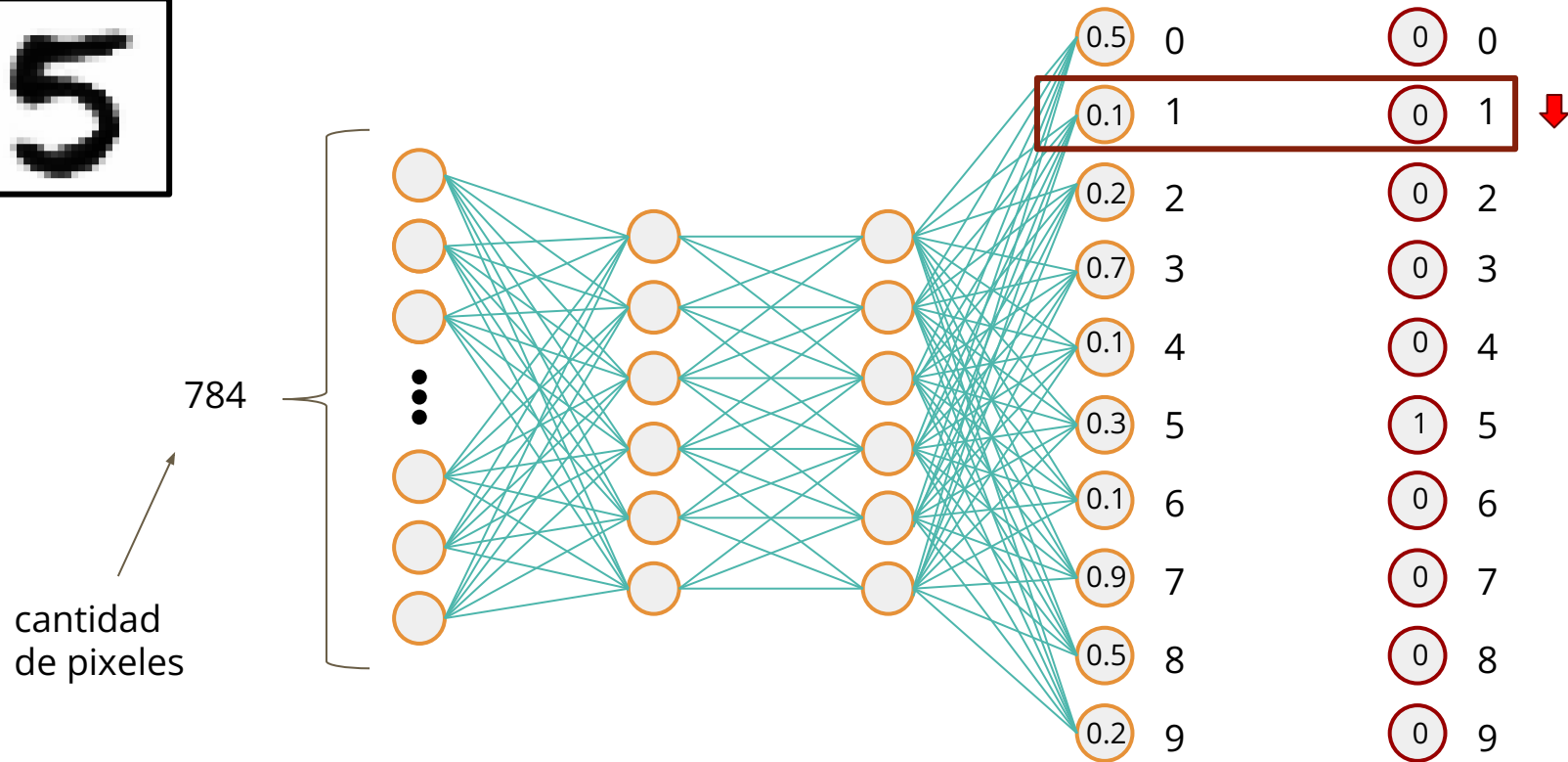
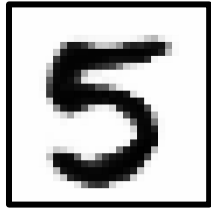


cantidad
de pixeles

784



Red Neuronal: Backpropagation o cómo calcular w y b



Red Neuronal: Backpropagation

Se define una función de costo que determina mi error a partir del valor predicho y el valor real.
Por ejemplo la función de pérdida cuadrática:

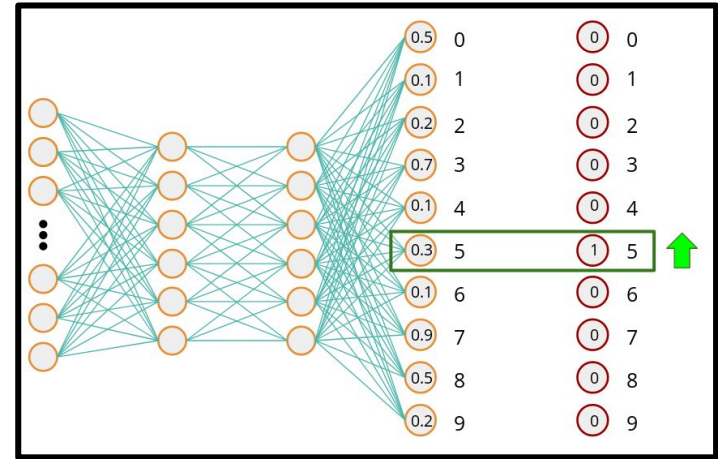
$$Cost = (f(x) - Y)^2$$

Donde el valor predicho $f(x)$ es:

$$f(x) = \sigma(z^l)$$

$$z^l = \sum_i w_i^l * a_i^{l-1} + b^l$$

Donde a_i^{l-1} corresponde al valor que toma la neurona i de la capa $l-1$.



Red Neuronal: Backpropagation

Para saber cuánto tengo que cambiar los valores, calculo cómo variaciones en w modifican la función de costo:

$$\frac{\partial C_0}{\partial w^l} = \frac{\partial z^l}{\partial w^l} * \frac{\partial a^l}{\partial z^l} * \frac{\partial C_0}{\partial a^l} = a^{l-1} * \sigma'(z^l) * 2(a^l - y)$$

Luego:

$$\frac{\partial C}{\partial w^l} = \frac{1}{n} \sum_{k=0}^{n-1} \frac{\partial C_k}{\partial w^l}$$

Equivalente para el bias.

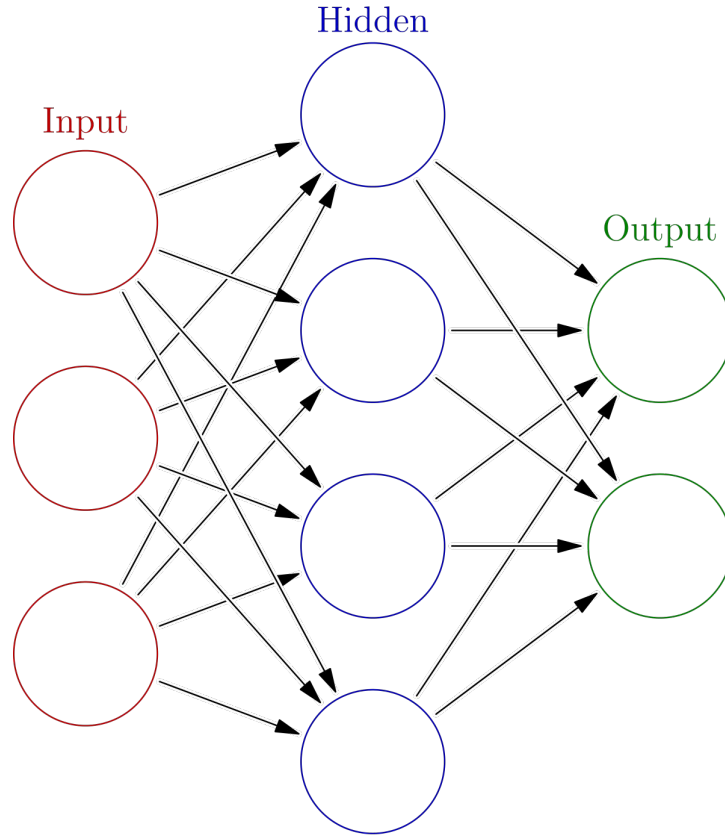
Red Neuronal: Backpropagation

Finalmente, para actualizar los valores de w hago:

$$W_i^+ = W_i - \eta * \frac{\partial C}{\partial w_i}$$

donde η es el “learning rate”.

Red Neuronal

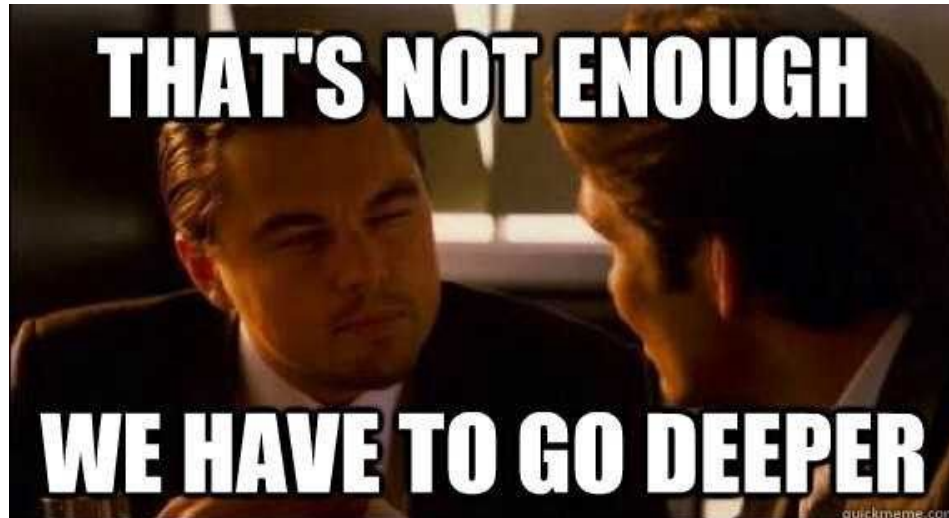


Un perceptrón simple es equivalente a una regresión logística.

Link

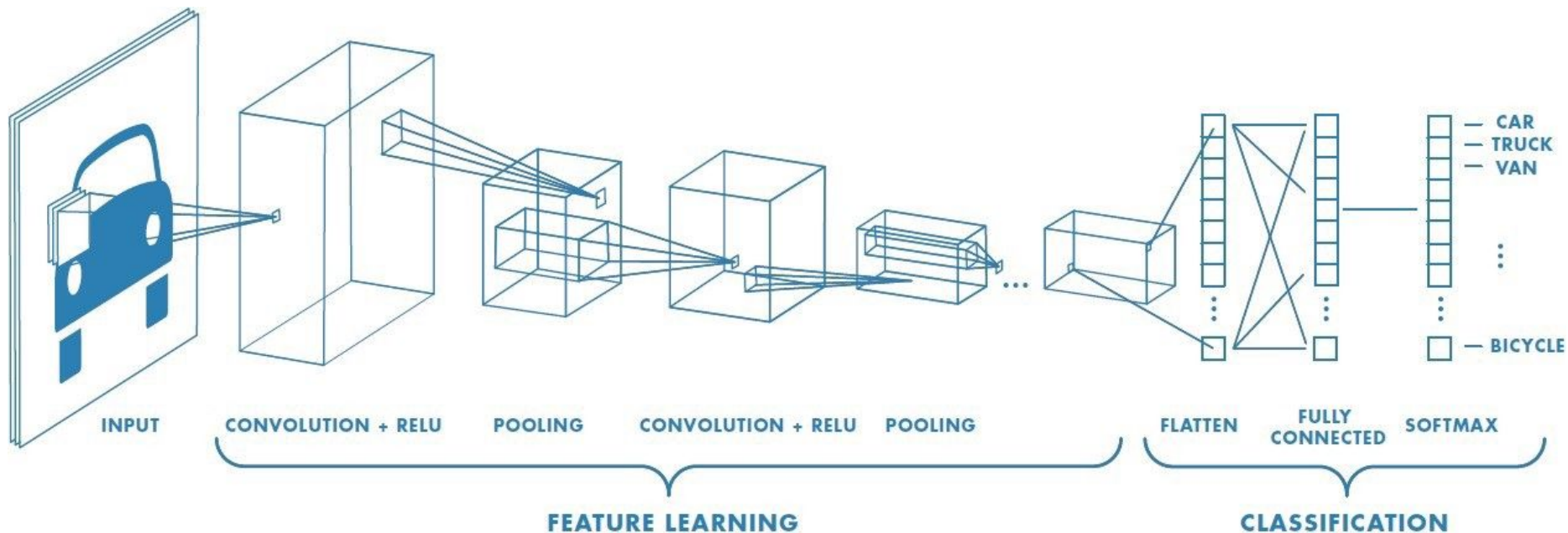
FIN ?

FIN ?



Convolutional Neural Networks

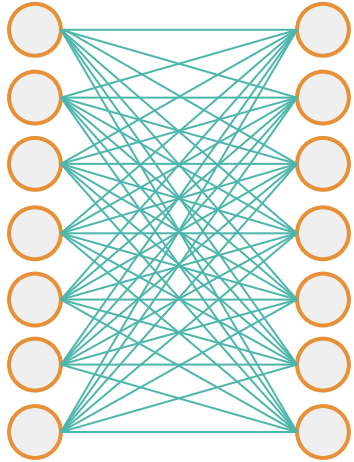
Convolutional Neural Networks: Estructura



Convolutional Neural Networks:

¿Qué son las CNN? Parte1: Convolución

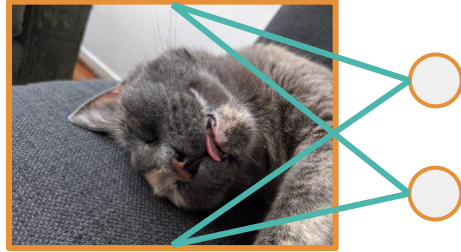
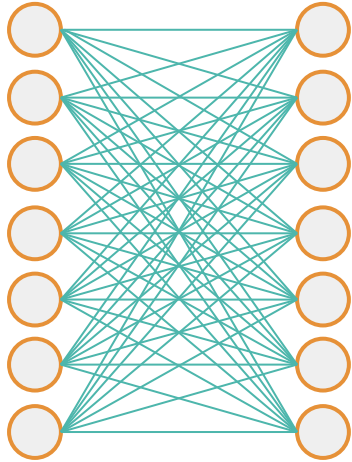
Antes (capa densa)



Convolutional Neural Networks:

¿Qué son las CNN? Parte1: Convolución

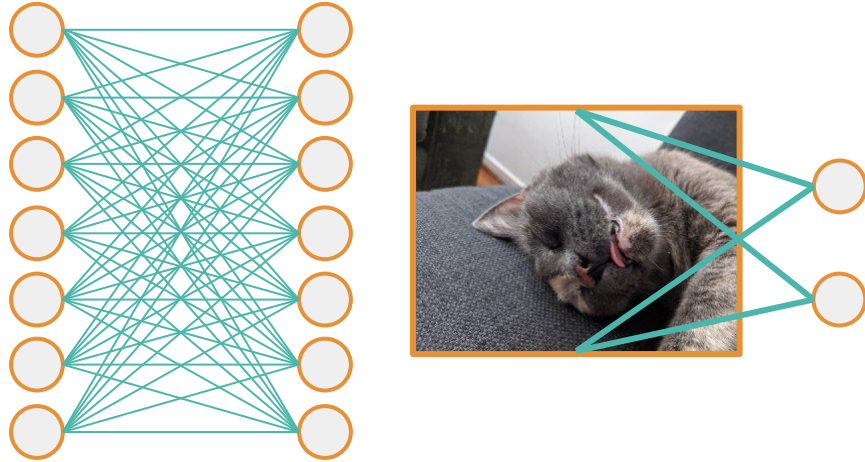
Antes (capa densa)



Convolutional Neural Networks:

¿Qué son las CNN? Parte1: Convolución

Antes (capa densa)

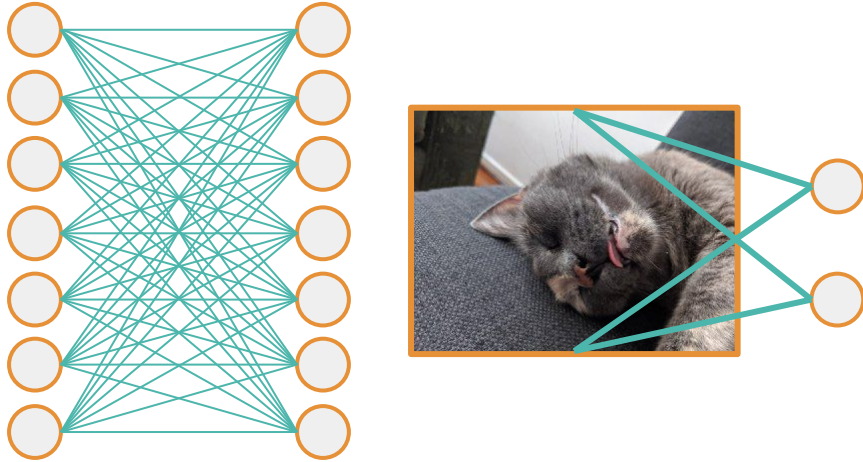


¡Muchos parámetros!

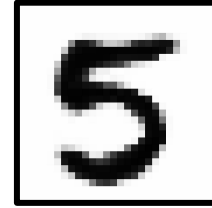
Convolutional Neural Networks:

¿Qué son las CNN? Parte1: Convolución

Antes (capa densa)



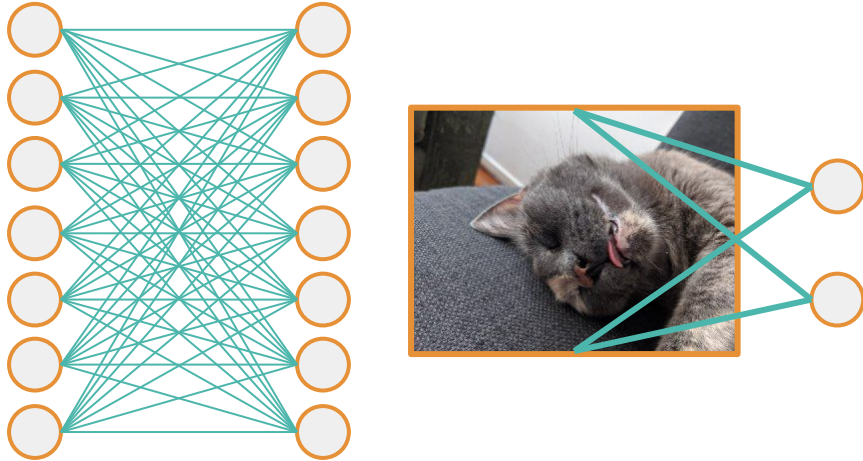
¡Muchos parámetros!



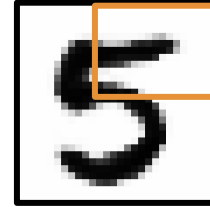
Convolutional Neural Networks:

¿Qué son las CNN? Parte1: Convolución

Antes (capa densa)



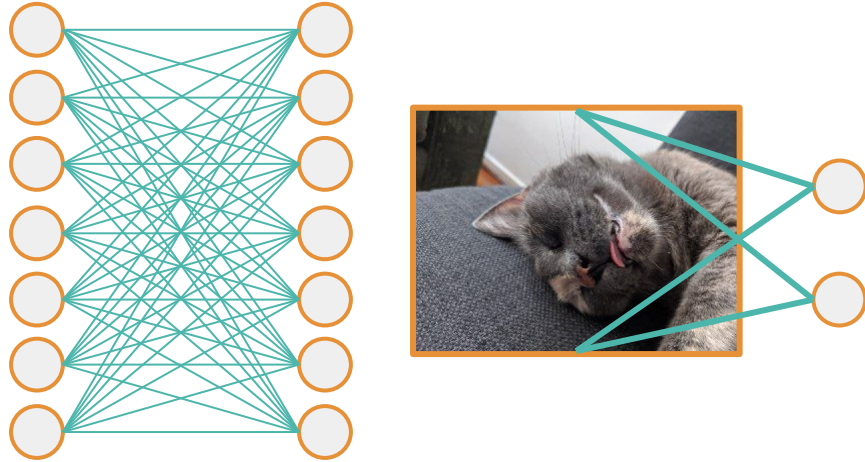
¡Muchos parámetros!



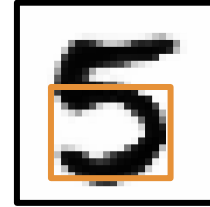
Convolutional Neural Networks:

¿Qué son las CNN? Parte1: Convolución

Antes (capa densa)



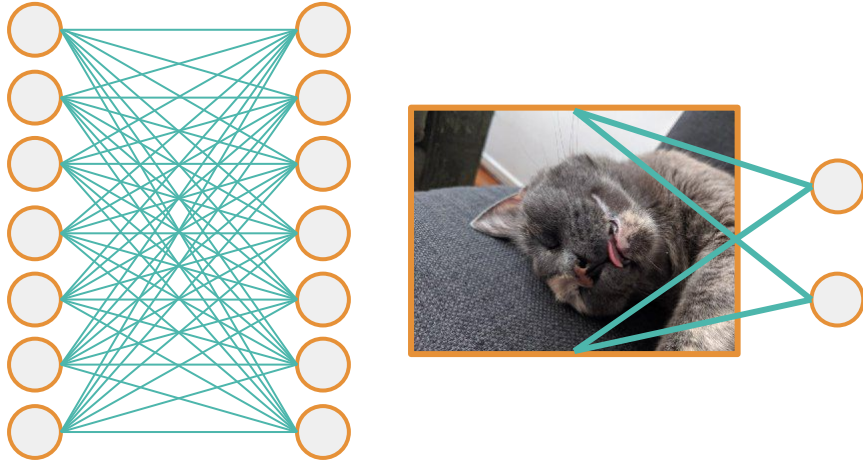
¡Muchos parámetros!



Convolutional Neural Networks:

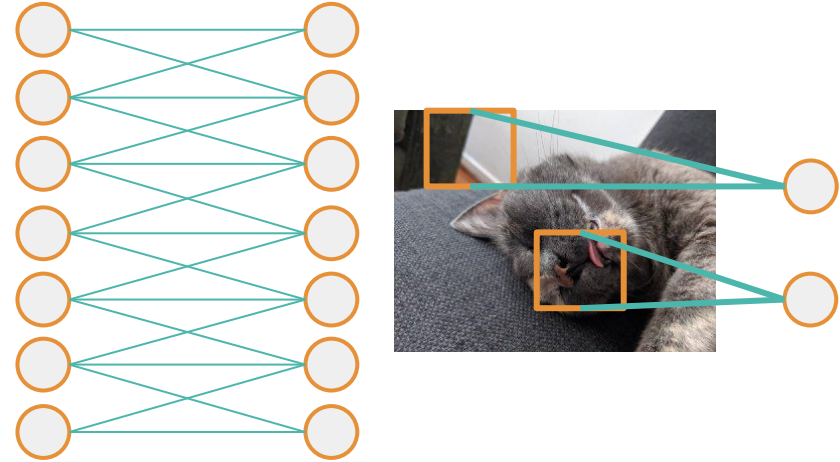
¿Qué son las CNN? Parte1: Convolución

Antes (capa densa)



¡Muchos parámetros!

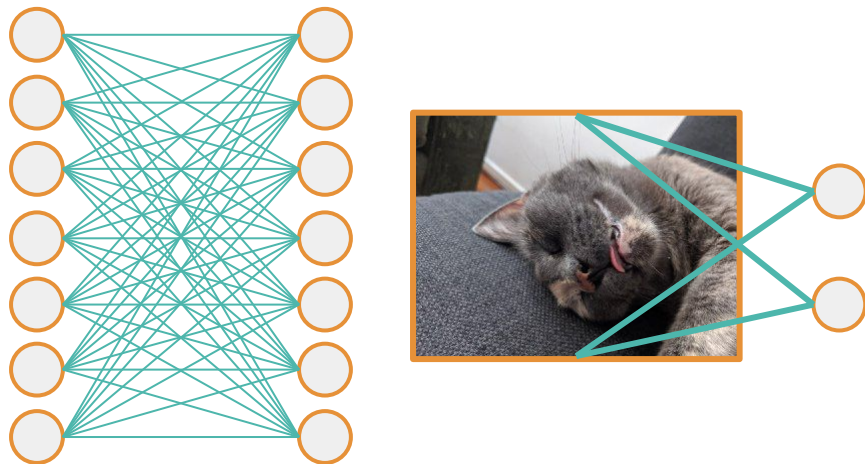
Convolución



Convolutional Neural Networks:

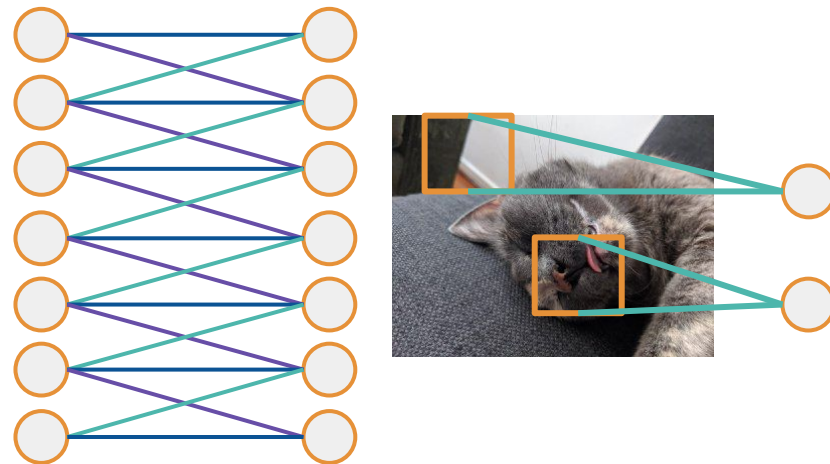
¿Qué son las CNN? Parte1: Convolución

Antes (capa densa)



¡Muchos parámetros!

Convolución

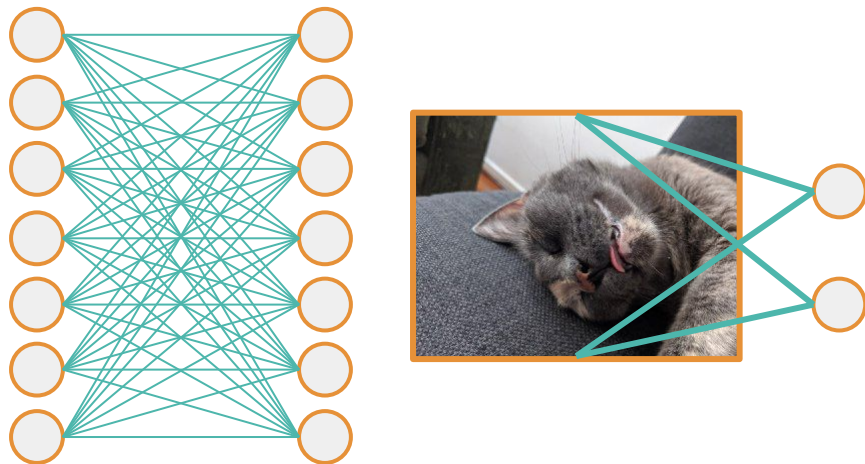


Menos parámetros → Generaliza mejor

Convolutional Neural Networks:

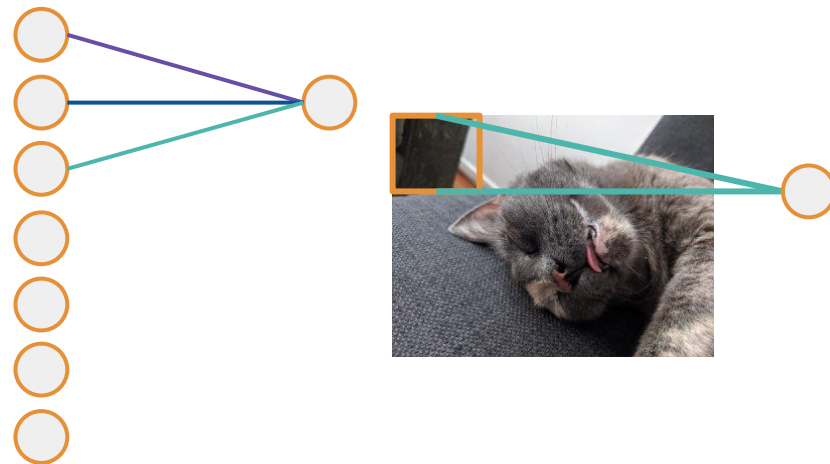
¿Qué son las CNN? Parte1: Convolución

Antes (capa densa)



¡Muchos parámetros!

Convolución

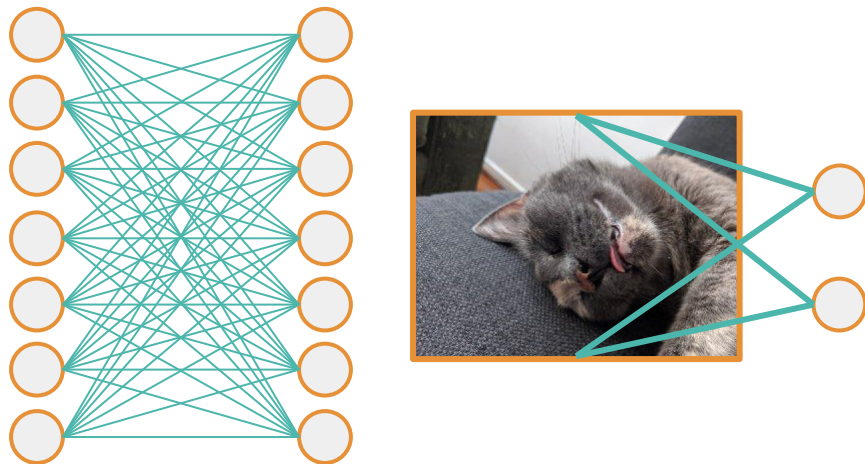


Menos parámetros → Generaliza mejor

Convolutional Neural Networks:

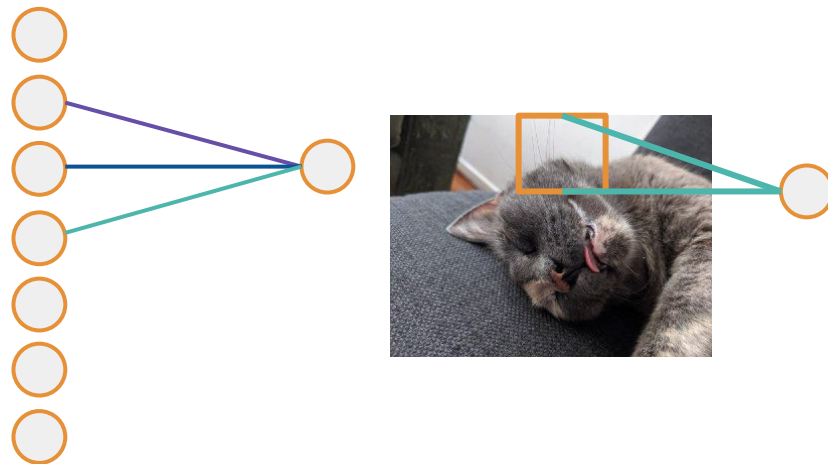
¿Qué son las CNN? Parte1: Convolución

Antes (capa densa)



¡Muchos parámetros!

Convolución

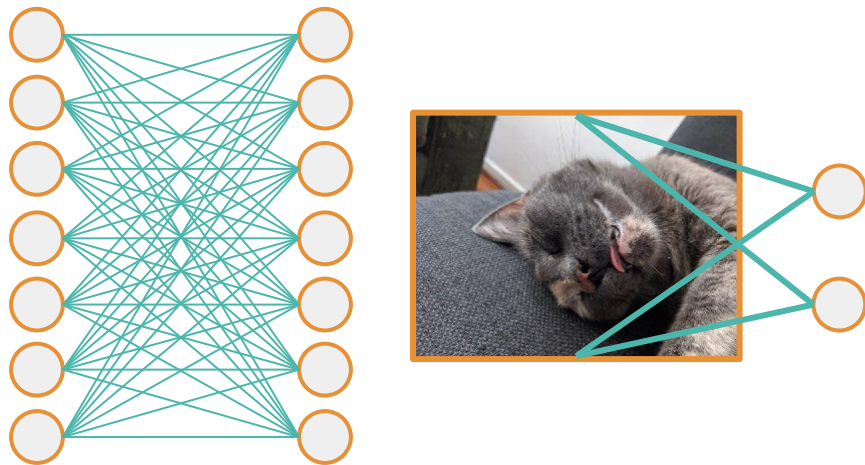


Menos parámetros → Generaliza mejor

Convolutional Neural Networks:

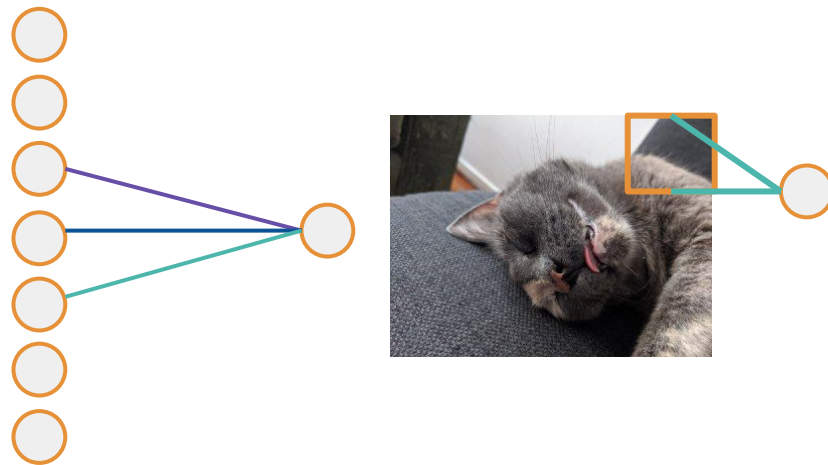
¿Qué son las CNN? Parte1: Convolución

Antes (capa densa)



¡Muchos parámetros!

Convolución

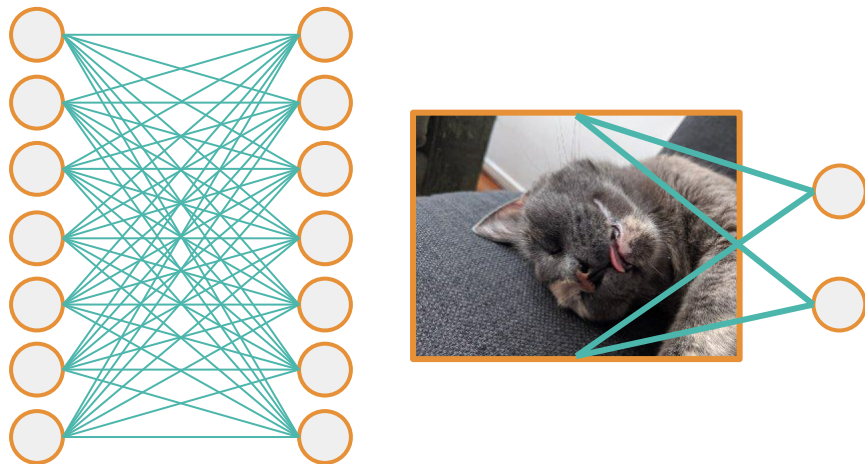


Menos parámetros → Generaliza mejor

Convolutional Neural Networks:

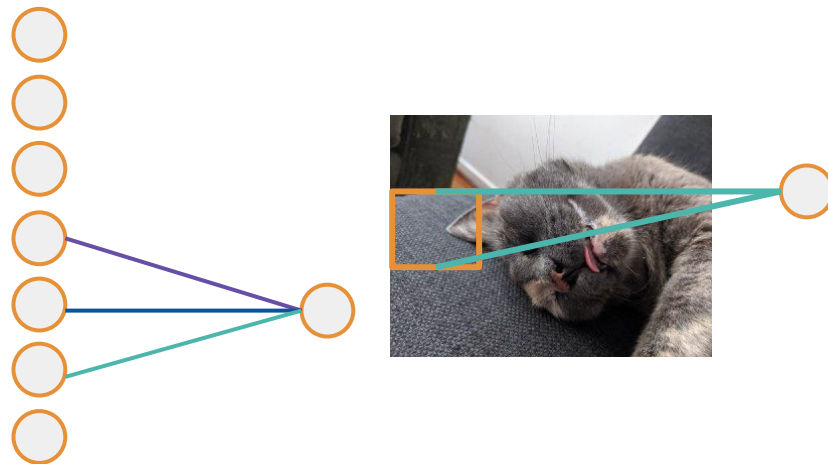
¿Qué son las CNN? Parte1: Convolución

Antes (capa densa)



¡Muchos parámetros!

Convolución

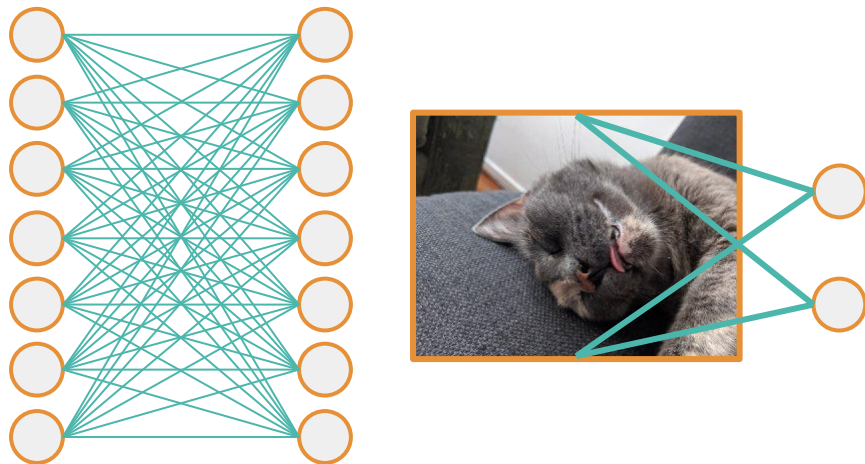


Menos parámetros → Generaliza mejor

Convolutional Neural Networks:

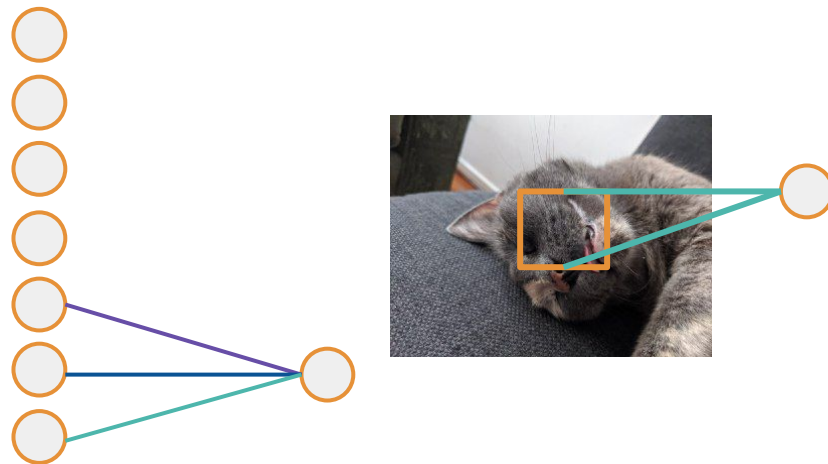
¿Qué son las CNN? Parte1: Convolución

Antes (capa densa)



¡Muchos parámetros!

Convolución



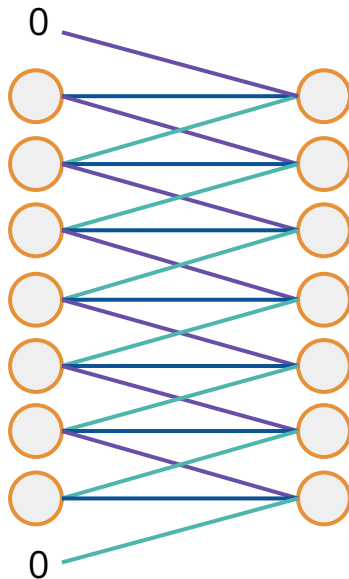
Menos parámetros → Generaliza mejor

Convolutional Neural Networks:

¿Qué son las CNN? Parte1: Convolución

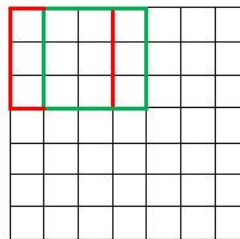
Padding: Agregar a los costados algo (generalmente ceros).

0	0	0	0	0	0	0
0						0
0						0
0						0
0						0
0						0
0	0	0	0	0	0	0

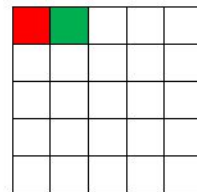


Stride: El número de pixels que se mueve la convolucional sobre la imagen..

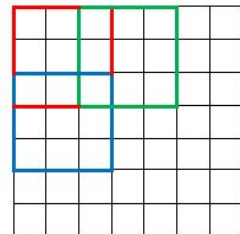
7 x 7 Input Volume



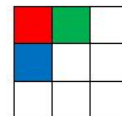
5 x 5 Output Volume



7 x 7 Input Volume



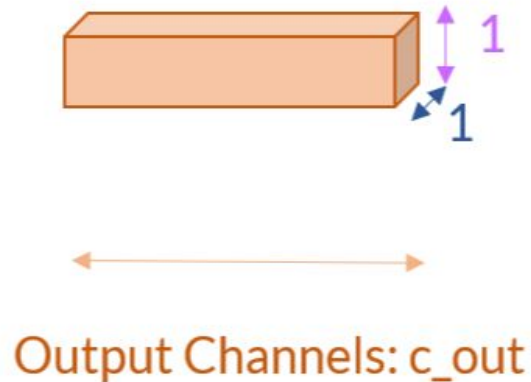
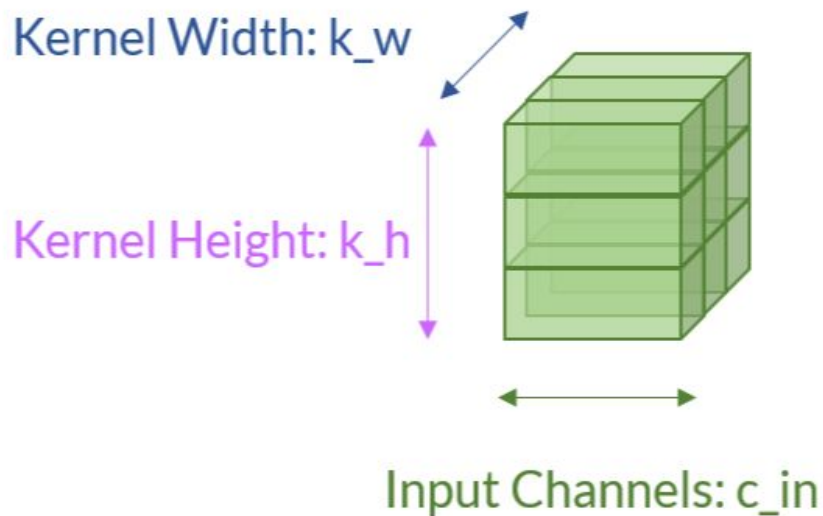
3 x 3 Output Volume



Convolutional Neural Networks:

¿Qué son las CNN? Parte1: Convolución

Kernel, Input Channels y Output Channels:



Convolutional Neural Networks:

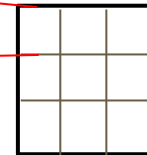
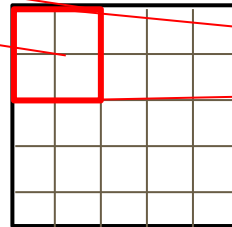
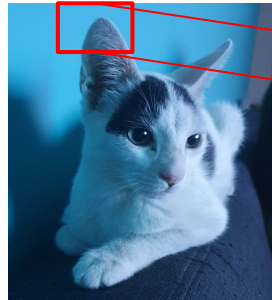
¿Qué son las CNN? Parte 2: Max Pooling

1	4	2	7
2	6	8	5
3	4	0	7
1	2	3	1

max pooling con
filtros 2x2 y
"stride" 2



6	8
4	7



Convolutional Neural Networks:

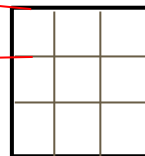
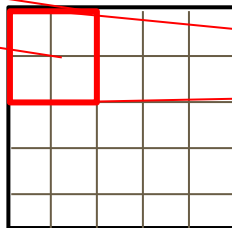
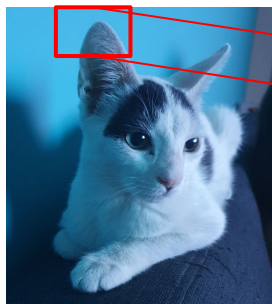
¿Qué son las CNN? Parte 2: Max Pooling

1	4	2	7
2	6	8	5
3	4	0	7
1	2	3	1

max pooling con
filtros 2x2 y
"stride" 2

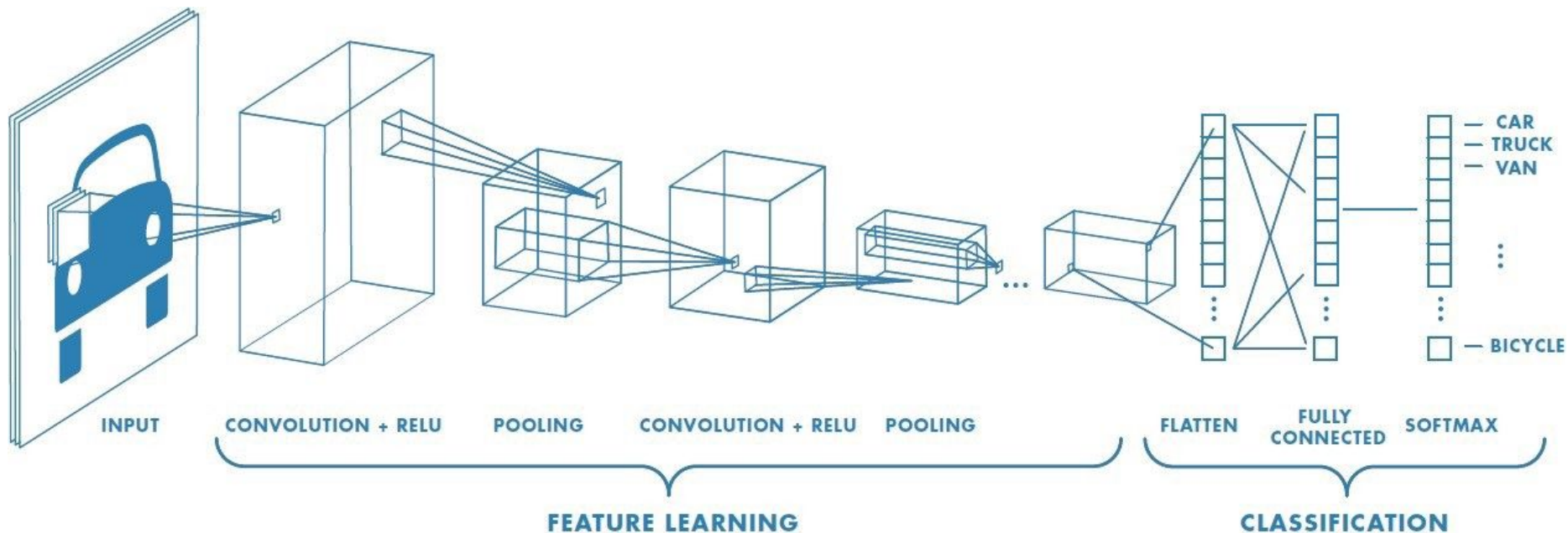


6	8
4	7

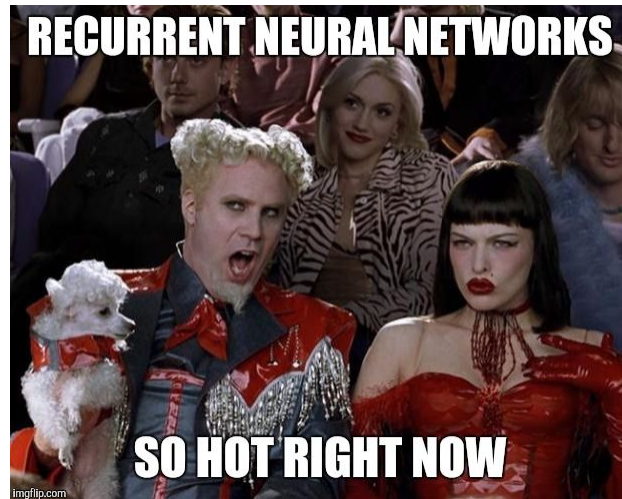


No necesariamente se tiene que quedar con el máximo, sino puede ser con el **promedio** u otras cosas más complicadas.

Convolutional Neural Networks:



Recurrent Neural Networks

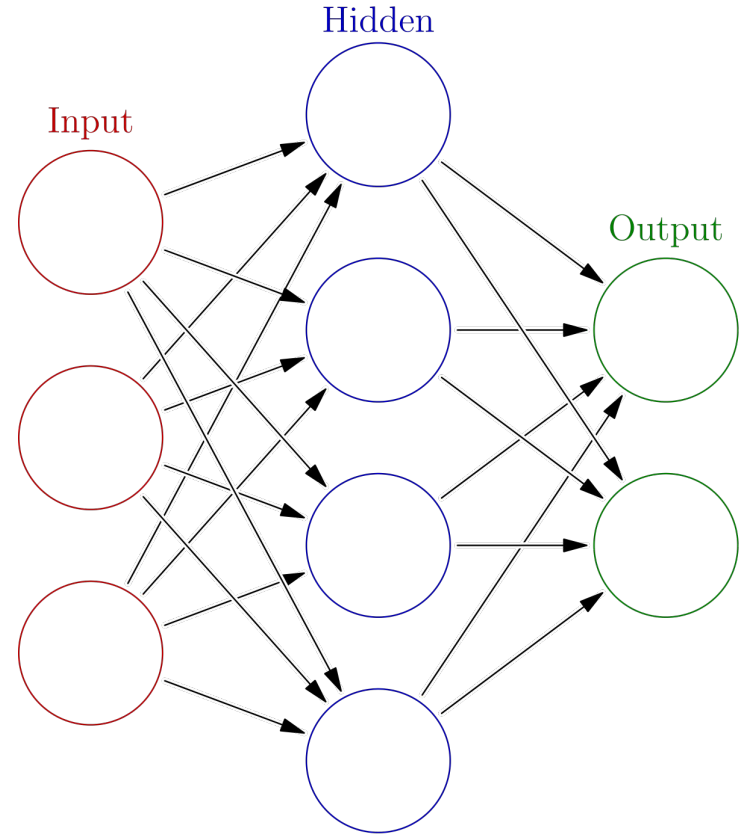


Recurrent Neural Networks

Antes

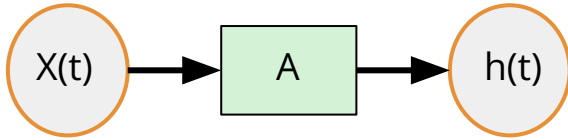


$$h(t) = \sigma(W \cdot x(t) + b)$$



Recurrent Neural Networks

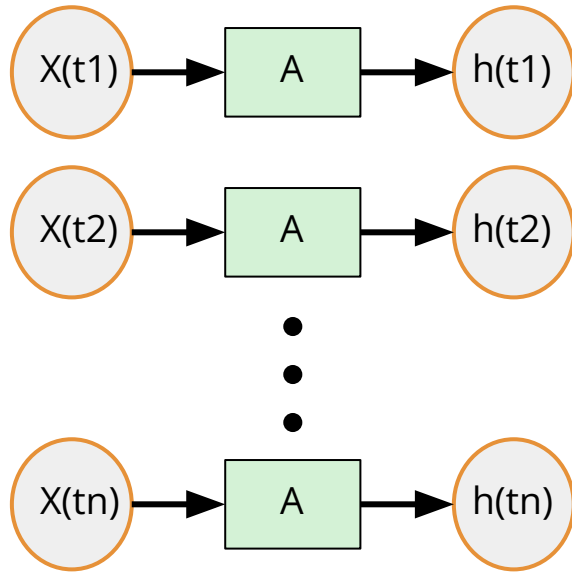
Antes



$$h(t) = \sigma(W \cdot x(t) + b)$$

Recurrent Neural Networks

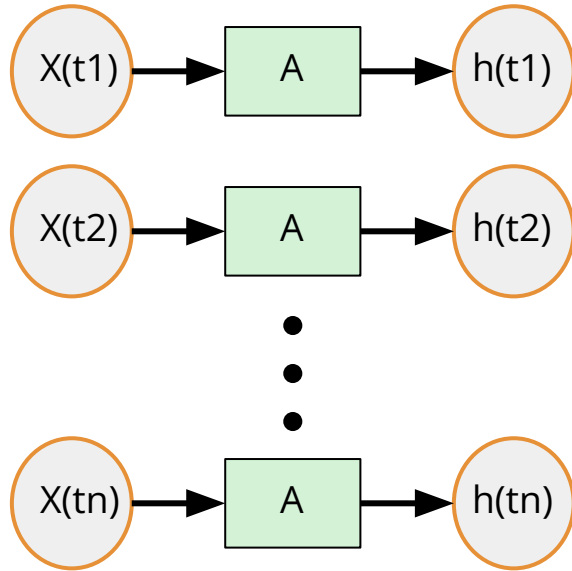
Antes



$$h(t) = \sigma(W \cdot x(t) + b)$$

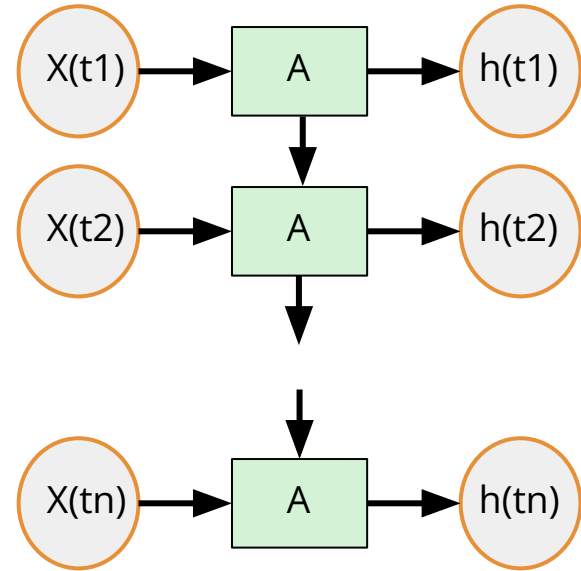
Recurrent Neural Networks

Antes

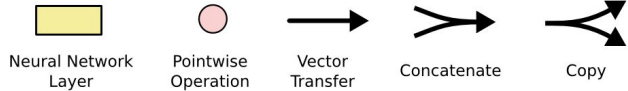
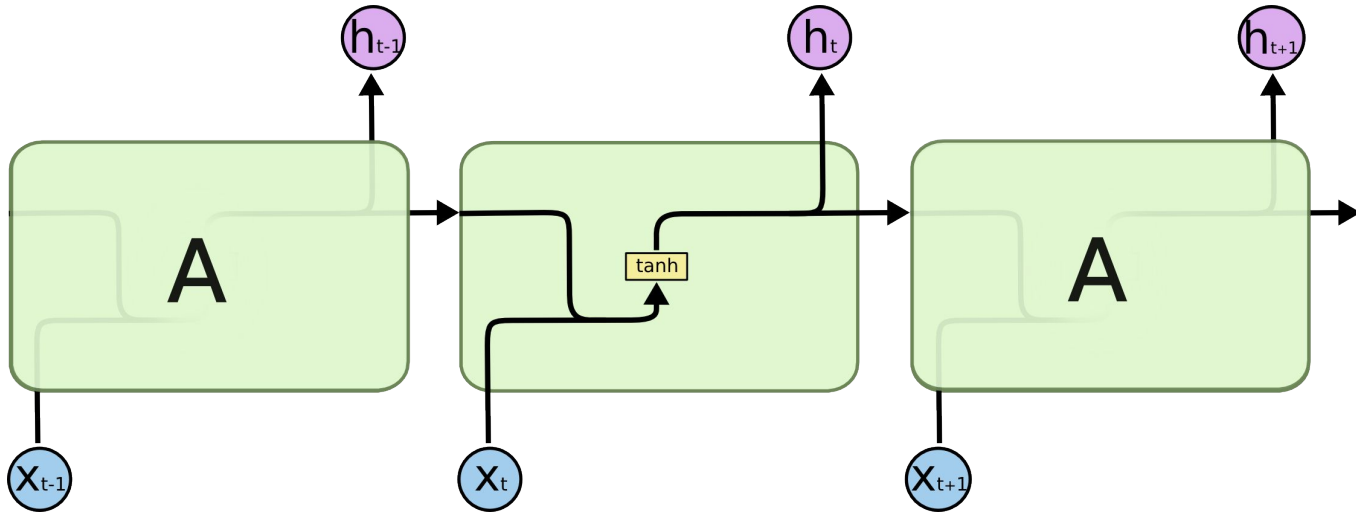


$$h(t) = \sigma(W \cdot x(t) + b)$$

RNN

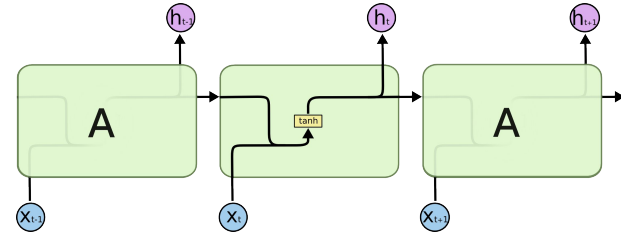


Recurrent Neural Networks



Recurrent Neural Networks

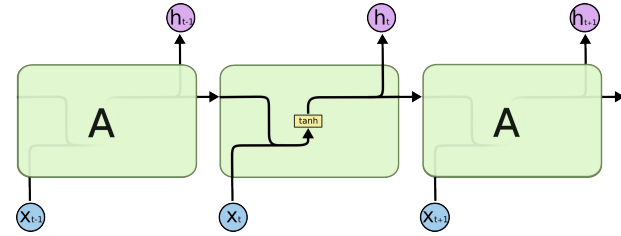
Problemas con este simple approach:



Recurrent Neural Networks

Problemas con este simple approach:

- Vanishing Gradients
- Exploding Gradients



Recurrent Neural Networks

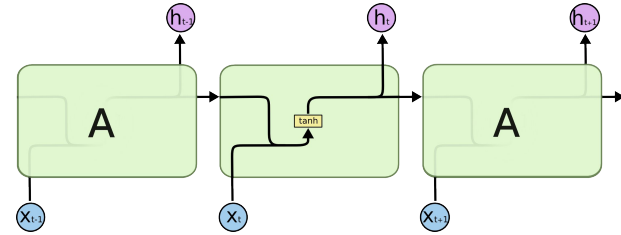
Problemas con este simple approach:

- Vanishing Gradients

$$0.99^{1000} = 0.000043...$$

- Exploding Gradients

$$1.01^{1000} = 20959.15...$$

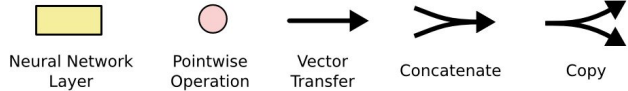
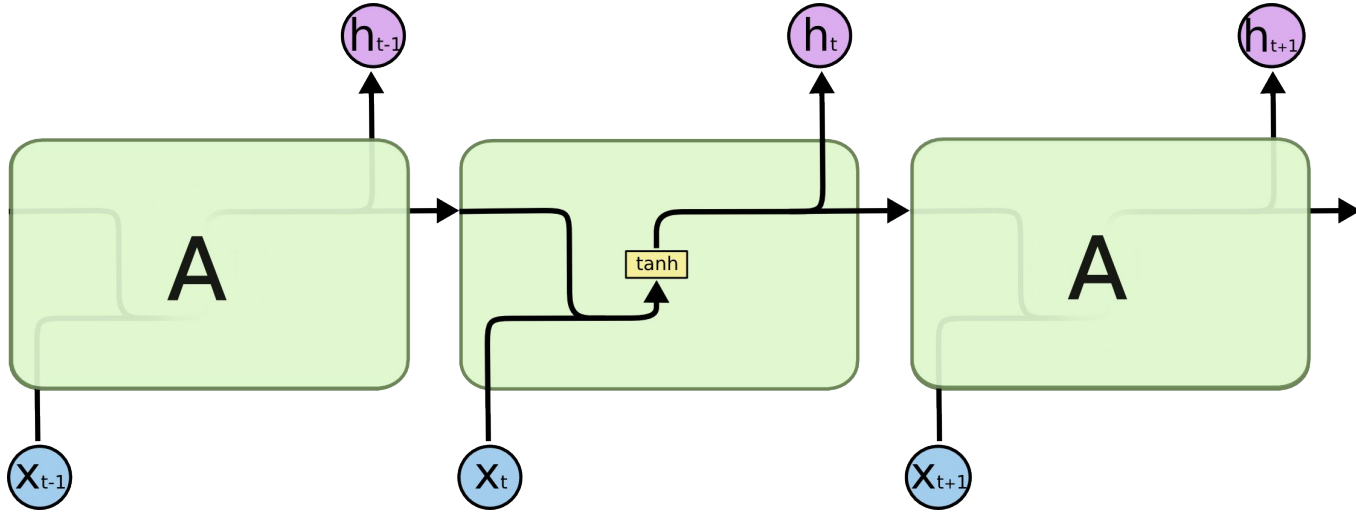




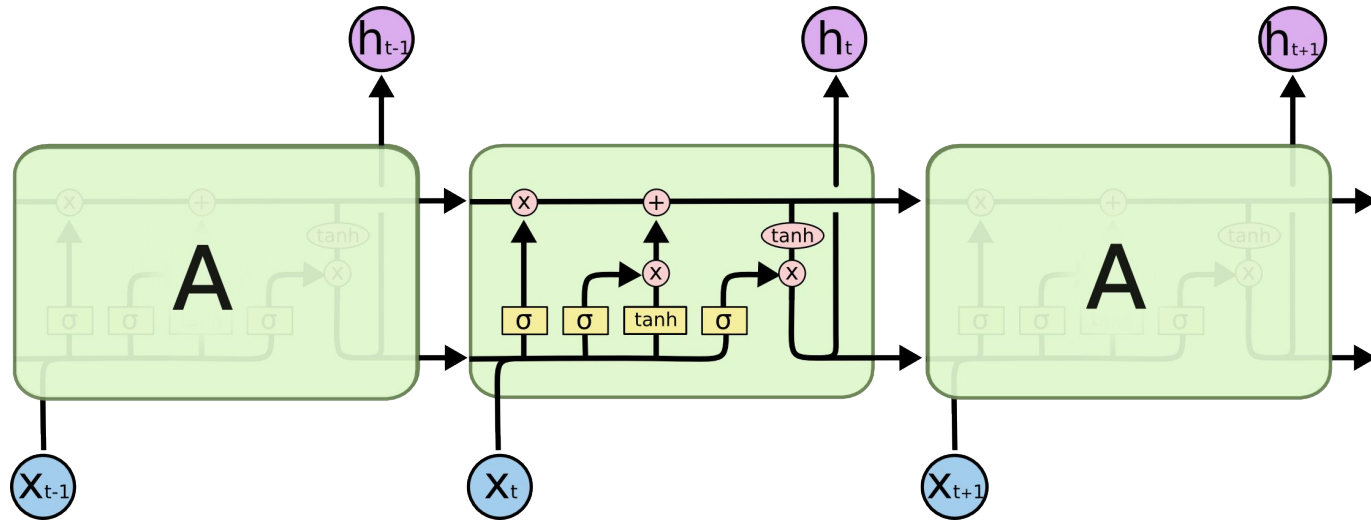
LSTM



Recurrent Neural Networks



Long Short Term Memory networks



Neural Network
Layer



Pointwise
Operation



Vector
Transfer

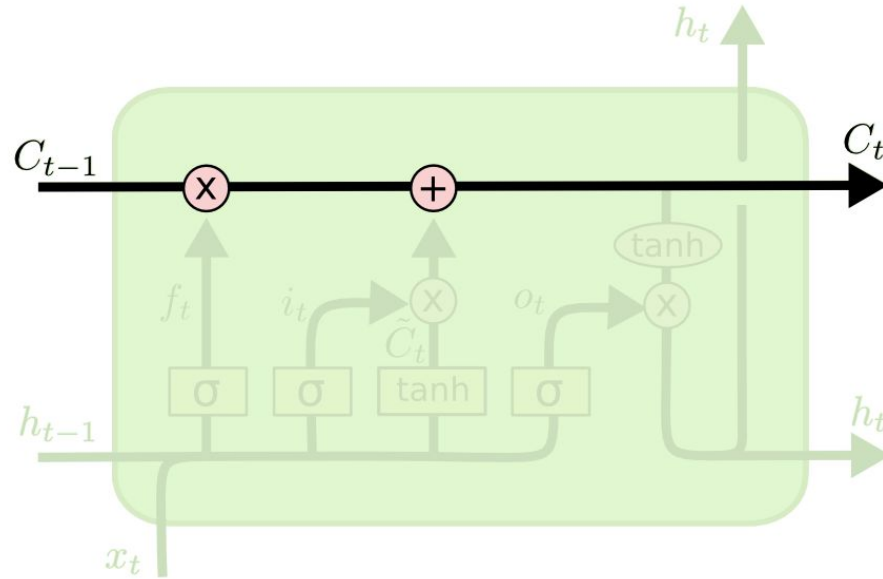


Concatenate



Copy

Long Short Term Memory networks



Neural Network
Layer



Pointwise
Operation



Vector
Transfer

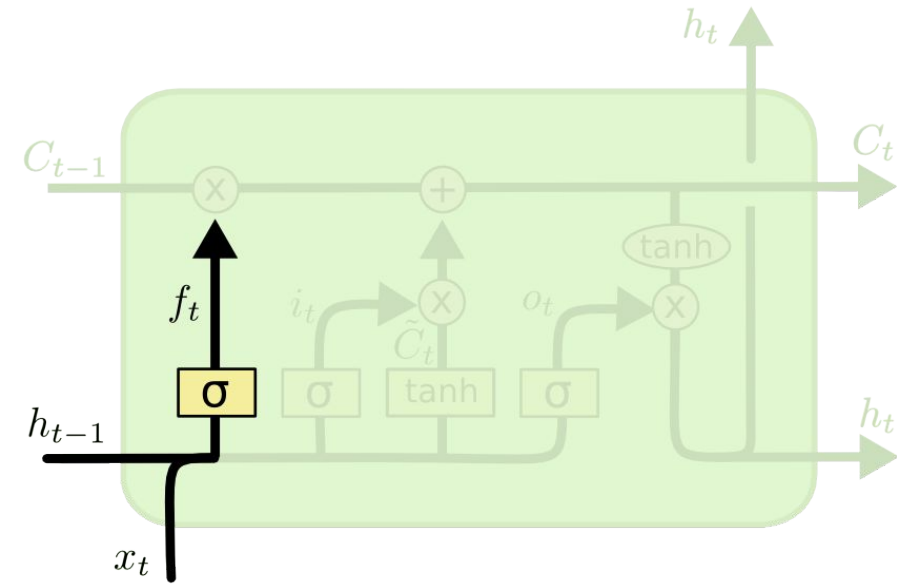


Concatenate

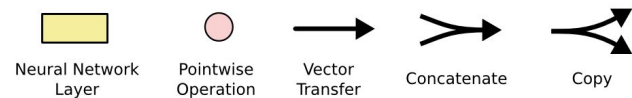


Copy

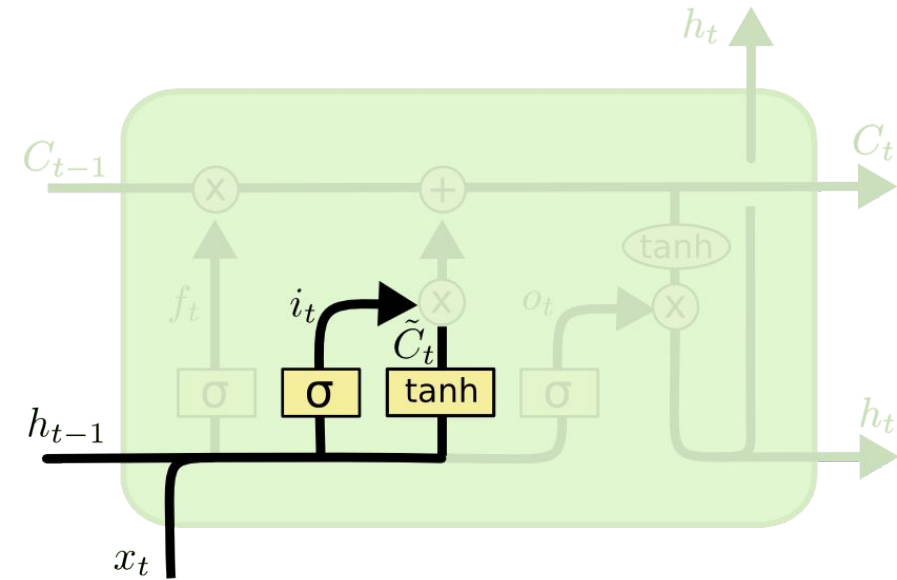
Long Short Term Memory networks



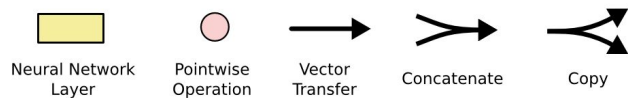
$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$



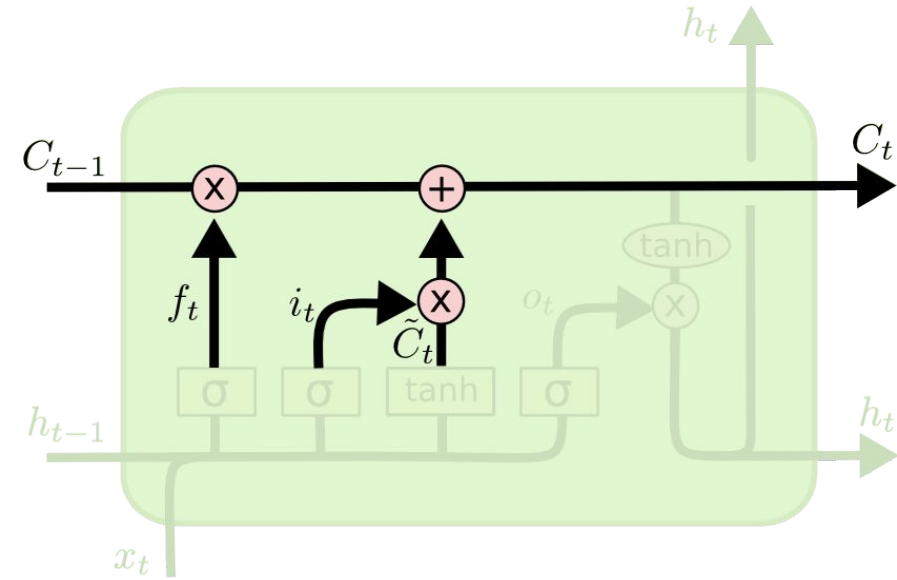
Long Short Term Memory networks



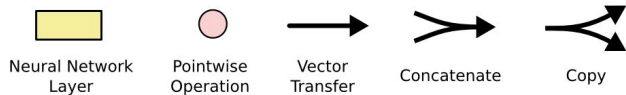
$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$
$$\tilde{c}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$



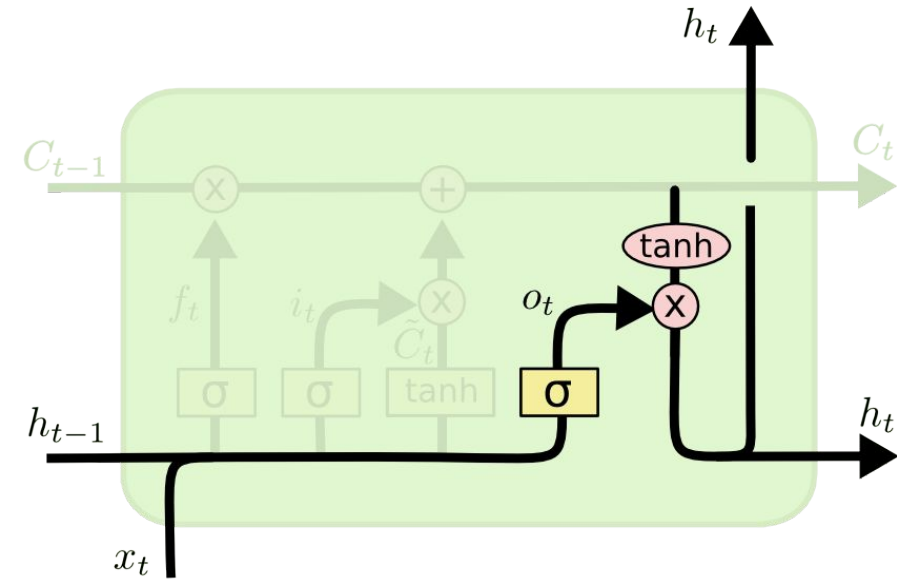
Long Short Term Memory networks



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

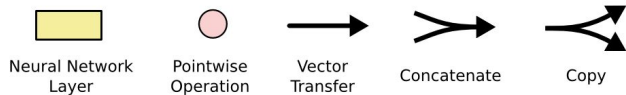


Long Short Term Memory networks



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$



Long Short Term Memory networks

- Conclusión simples:
 - Las LSTM tienen en cuenta la estructura ordenada de la serie de tiempo.
 - Las LSTM son buenas para aprender dependencias distantes en el tiempo.

Prophet

The image features a minimalist design with the word "Prophet" centered in a bold, orange, sans-serif font. The text is framed by two horizontal teal bars at the top and bottom, each accompanied by a thinner light blue line. Two short, thick, olive-green bars are positioned horizontally on either side of the word, aligned with its vertical center.

Prophet

Descompone en 3 a la serie de tiempo:

$$y(t) = g(t) + s(t) + h(t) + e_t$$

Prophet

Descompone en 3 a la serie de tiempo:

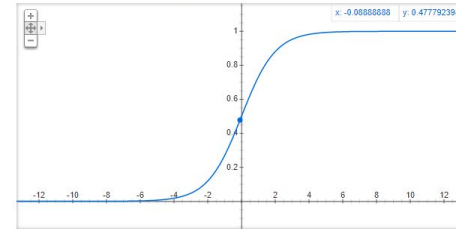
$$y(t) = g(t) + s(t) + h(t) + e_t$$

- $g(t)$: la tendencia de la serie temporal (trend). Modela cambios no periódicos.

$$g(t) = \frac{C}{1 + \exp(-k(t-m))}$$

(la fórmula de $g(t)$ una forma simplificada de lo que usan en verdad)

Gráfico de $1/(1+e^{-x})$



Prophet

Descompone en 3 a la serie de tiempo:

$$y(t) = g(t) + s(t) + h(t) + e_t$$

- $g(t)$: la tendencia de la serie temporal (trend). Modela cambios no periódicos.
- $s(t)$: la periodicidad de la serie temporal (seasonality)

$$s(t) = \Sigma \left(a_n \cos\left(\frac{2\pi nt}{P}\right) + b_n \sin\left(\frac{2\pi nt}{P}\right) \right)$$



Prophet

Descompone en 3 a la serie de tiempo:

$$y(t) = g(t) + s(t) + h(t) + e_t$$

- $g(t)$: la tendencia de la serie temporal (trend). Modela cambios no periódicos.
- $s(t)$: la periodicidad de la serie temporal (seasonality)
- $h(t)$: las irregularidades de la serie temporal(holidays)

$$h(t) = \kappa[1(t \in D_1), \dots, 1(t \in D_L)]$$

Prophet

Descompone en 3 a la serie de tiempo:

$$y(t) = g(t) + s(t) + h(t) + e_t$$

- $g(t)$: la tendencia de la serie temporal (trend). Modela cambios no periódicos.
- $s(t)$: la periodicidad de la serie temporal (seasonality)
- $h(t)$: las irregularidades de la serie temporal(holidays)
- e_t : el error (que se asume gaussiano)

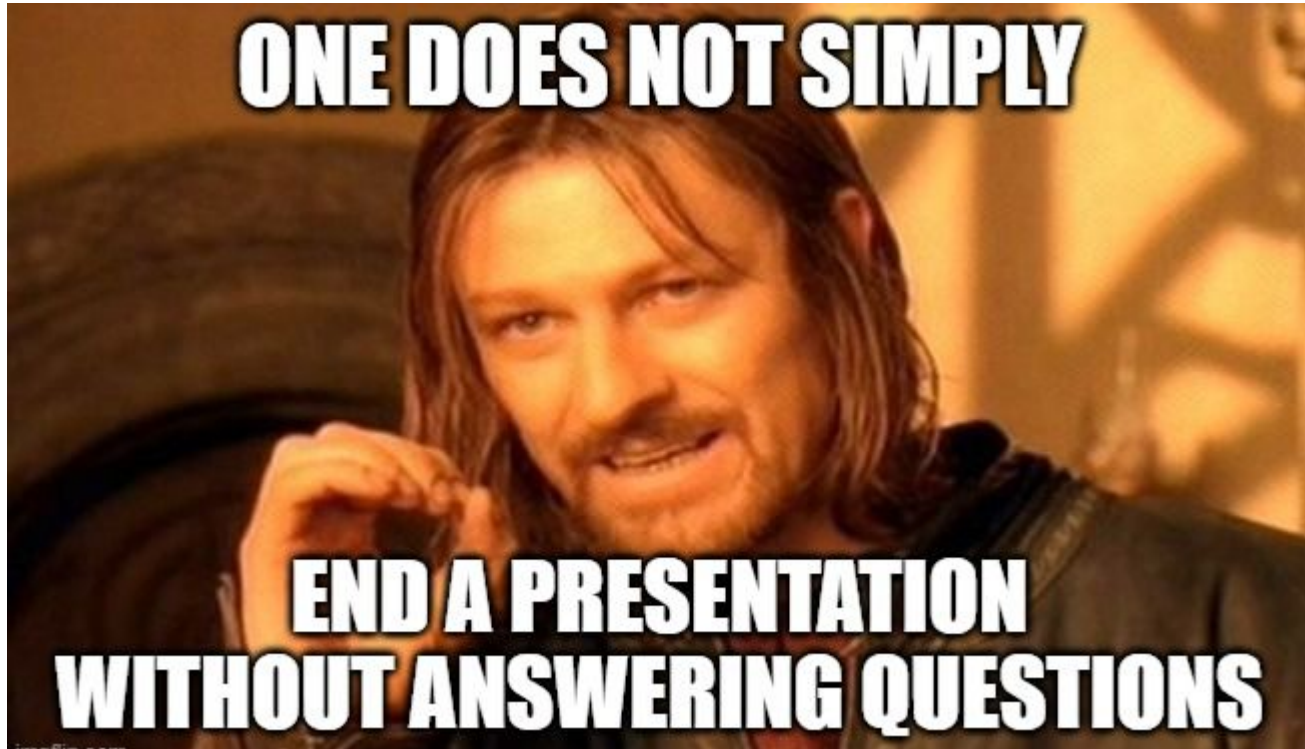
Prophet

Descompone en 3 a la serie de tiempo:

$$y(t) = g(t) + s(t) + h(t) + e_t$$

Luego usa L-BFGS (un algoritmo de optimización de la familia del método de Newton) para calcular las distribuciones a posteriori de los parámetros.

Preguntas



Código

Código: 1D CNN

Importamos

```
[ ] import torch
import torch.nn as nn
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error
%matplotlib inline
```

Código: 1D CNN

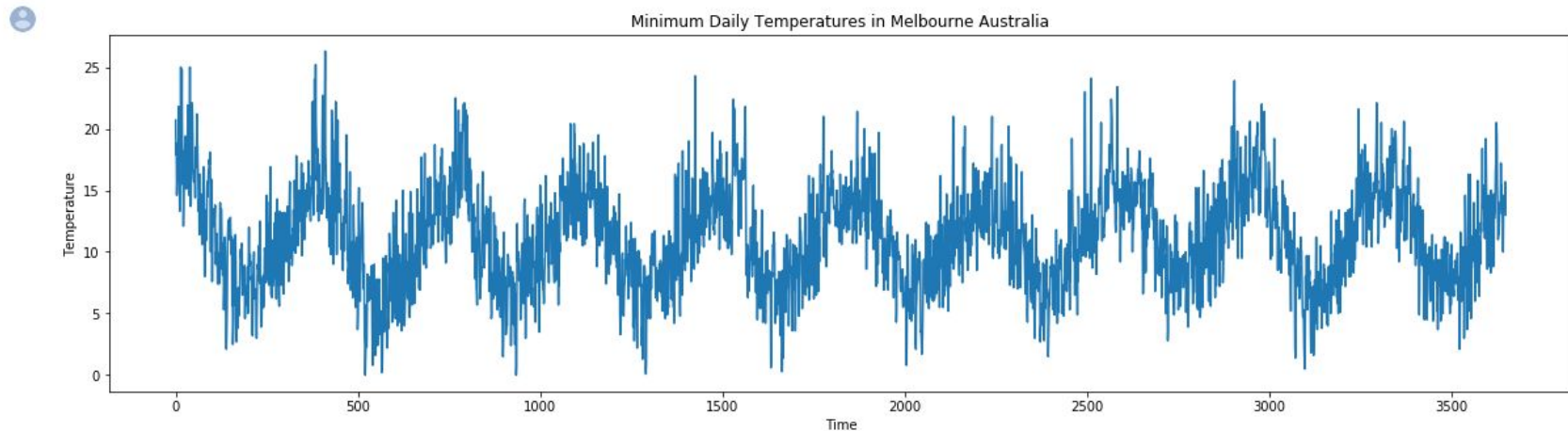
Importamos los datos

```
▶ data = pd.read_csv("daily_minimum_temperatures.csv", sep=",")  
print(data.shape)  
data.head()
```

Código: 1D CNN

visualizamos los datos

```
plt.figure(figsize = [20,5])  
plt.plot(data.Temp)  
plt.title("Minimum Daily Temperatures in Melbourne Australia")  
plt.xlabel("Time")  
plt.ylabel("Temperature")  
plt.show()
```



Código: 1D CNN

Separamos en train y test

```
[ ] train_percentage = 0.67
    train_size = int(len(data)*train_percentage)

    train = data["Temp"][:train_size].values.astype(float)
    test = data["Temp"][train_size:].values.astype(float)

    sc = MinMaxScaler(feature_range=(-1, 1))
    train_norm = sc.fit_transform(train.reshape(-1, 1))

    train_norm = torch.FloatTensor(train_norm).view(-1)
```

Código: 1D CNN

Preparamos el dataset para la red neuronal

```
[ ] def create_input_output(input_data, window, ndays = 1):  
    input_output = []  
    data_size = len(input_data)  
    for i in range(data_size - window - ndays):  
        train_seq = input_data[i:i+window]  
        train_label = input_data[i+window+ndays:i+window+ndays+1]  
        input_output.append((train_seq ,train_label))  
    return input_output  
ndays = 10  
window = 30  
train_input_output = create_input_output(train_norm, window, ndays)  
train_input_output[:2]
```

```
[(tensor([ 0.5741,  0.3612,  0.4297,  0.1103,  0.2015,  0.2015,  0.2015,  0.3232,  
          0.6578,  0.5209,  0.2319,  0.0114,  0.2700,  0.6350,  0.9011,  0.5741,  
          0.5665,  0.8859,  0.3460,  0.1787,  0.3840, -0.0798,  0.0951,  0.2167,  
          0.2548,  0.4221,  0.4753,  0.3080,  0.1787,  0.1483])),  
 tensor([0.1407])),  
 (tensor([ 0.3612,  0.4297,  0.1103,  0.2015,  0.2015,  0.2015,  0.3232,  0.6578,  
          0.5209,  0.2319,  0.0114,  0.2700,  0.6350,  0.9011,  0.5741,  0.5665,  
          0.8859,  0.3460,  0.1787,  0.3840, -0.0798,  0.0951,  0.2167,  0.2548,  
          0.4221,  0.4753,  0.3080,  0.1787,  0.1483,  0.1711])),  
 tensor([0.0418]))]
```

Código: 1D CNN

Definimos la red neuronal

```
# Documentation of pytorch Conv1D: https://pytorch.org/docs/stable/generated/torch.nn.Conv1d.html
class NN_conv(nn.Module):
    def __init__(self):
        super(NN_conv, self).__init__()
        self.conv1 = nn.Sequential(
            nn.Conv1d(
                in_channels=1,      # Number of channels in the input image
                out_channels=5,     # Number of channels produced by the convolution
                kernel_size=5,      # Size of the convolving kernel
                stride=1,           # Stride of the convolution. Default: 1
                padding=2,          # Zero-padding added to both sides of the input. Default: 0
            ),
            nn.ReLU(),
            nn.MaxPool1d(kernel_size=3),
        )
        self.conv2 = nn.Sequential(
            nn.Conv1d(
                in_channels=5,
                out_channels=8,
                kernel_size=5,
                stride=1,
                padding=2,
            ),
            nn.ReLU(),
            nn.MaxPool1d(kernel_size=2),
        )
        self.out = nn.Linear(40, 1)

    def forward(self, x):
        #print(x.size())
        x = self.conv1(x)
        #print(x.size())
        x = self.conv2(x)
        #print(x.size())
        x = x.view(-1, 8 * 5)
        #print(x.size())
        output = self.out(x)
        return output
```

Código: 1D CNN

Definimos el modelo

```
[ ] model = NN_conv()  
    loss_function = nn.MSELoss()  
    optimizer = torch.optim.Adam(model.parameters(), lr=0.001) #torch.optim.SGD(model.parameters(), lr=0.001)
```

Código: 1D CNN

Realizamos las predicciones

```
▶ test_norm = sc.transform(test.reshape(-1, 1))  
test_norm = torch.FloatTensor(test_norm).view(-1)  
test_input_output = create_input_output(test_norm, window, ndays)  
test_input_output[:2]
```

```
[ ] y_pred_norm = []  
for inputs, labels in test_input_output:  
    with torch.no_grad():  
        inputs = inputs.view(1,1>window)  
        y_pred_norm.append(model(inputs).item())  
y_pred_norm[:5]
```

Código: 1D CNN

Medimos el error

```
 y_true = test>window+ndays:]
```

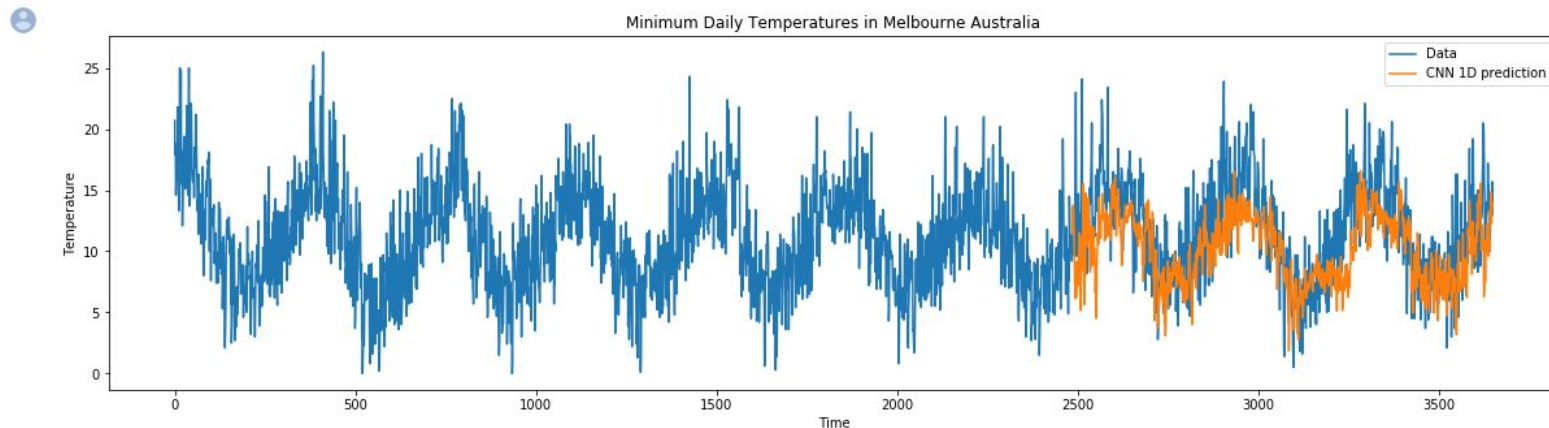
```
[ ] mean_squared_error(y_true, y_pred)
```

```
13.524712006584169
```

Código: 1D CNN

Mostramos resultados

```
plt.figure(figsize = [20,5])
plt.plot(data.Temp, label="Data")
plt.plot(range(len(train)+window+ndays, len(train) + len(test)), y_pred, label="CNN 1D prediction")
plt.title("Minimum Daily Temperatures in Melbourne Australia")
plt.xlabel("Time")
plt.ylabel("Temperature")
plt.legend()
plt.show()
```



Código: LSTM

Código: LSTM

Definimos la red neuronal

```
[ ] class LSTM(nn.Module):  
    def __init__(self, input_size=1, hidden_layer_size=100, output_size=1):  
        super().__init__()  
        self.hidden_layer_size = hidden_layer_size  
  
        self.lstm = nn.LSTM(input_size, hidden_layer_size)  
  
        self.linear = nn.Linear(hidden_layer_size, output_size)  
  
    def forward(self, input_seq):  
        lstm_out, _ = self.lstm(input_seq.view(len(input_seq) ,1, -1))  
        predictions = self.linear(lstm_out.view(len(input_seq), -1))  
        return predictions[-1]
```

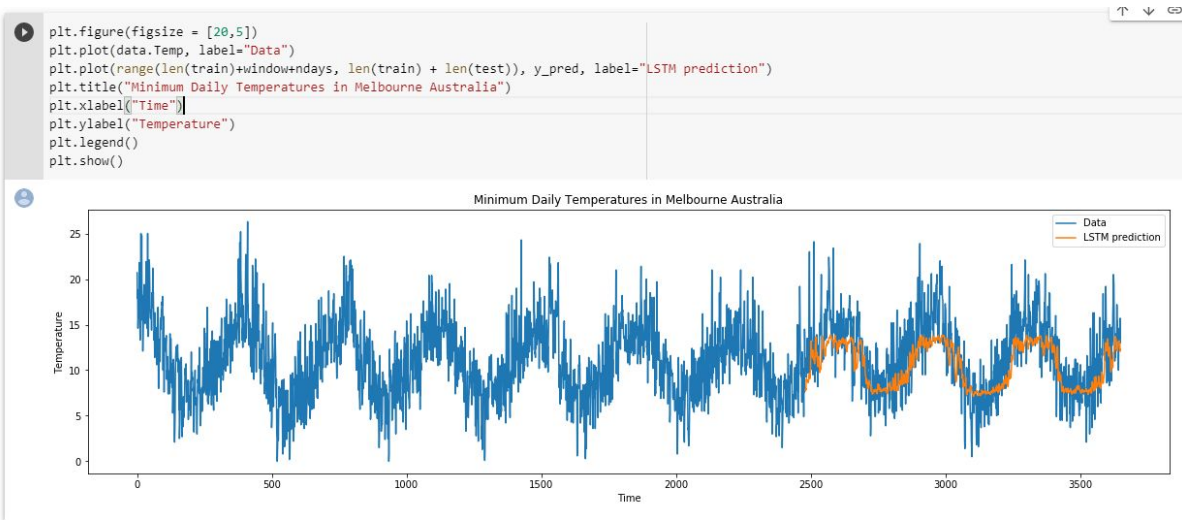
Código: LSTM

Medimos el error

```
[ ] mean_squared_error(y_true, y_pred)
```

9.528078730487337

Visualizamos los resultados



Código: Prophet

Código: Prophet

Importamos

```
[ ] from fbprophet import Prophet
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error
%matplotlib inline
```

Código: Prophet

Entrenamos el modelo

```
[ ] prophet_basic = Prophet()  
    prophet_basic.fit(train)
```

Código: Prophet

Realizamos la predicción

```
[ ] predictions = prophet_basic.predict(predictions)
predictions.tail(5)
```



	ds	trend	yhat_lower	yhat_upper	trend_lower	trend_upper	additive_terms	additive_terms_lower	additive_terms_upper	weekly	weekly_lower	weekly_upper	
3646	1990-12-27	10.366014	10.010077	17.009555	9.824297	10.872548	3.108427	3.108427	3.108427	-0.163845	-0.163845	-0.163845	3
3647	1990-12-28	10.365739	10.160343	16.954099	9.823508	10.872655	3.317855	3.317855	3.317855	-0.032438	-0.032438	-0.032438	3
3648	1990-12-29	10.365465	10.141042	17.296982	9.822719	10.872762	3.386761	3.386761	3.386761	-0.037599	-0.037599	-0.037599	3
3649	1990-12-30	10.365190	10.325846	17.063082	9.821932	10.872851	3.301382	3.301382	3.301382	-0.192036	-0.192036	-0.192036	3
3650	1990-12-31	10.364915	10.569192	17.794253	9.821147	10.872934	3.610559	3.610559	3.610559	0.054040	0.054040	0.054040	3

```
[ ] test['ds'] = pd.to_datetime(test['ds'])
results = pd.merge(test, predictions[["ds", "yhat"]], on = ["ds"], how='left')
```

Código: Prophet

Medimos el error

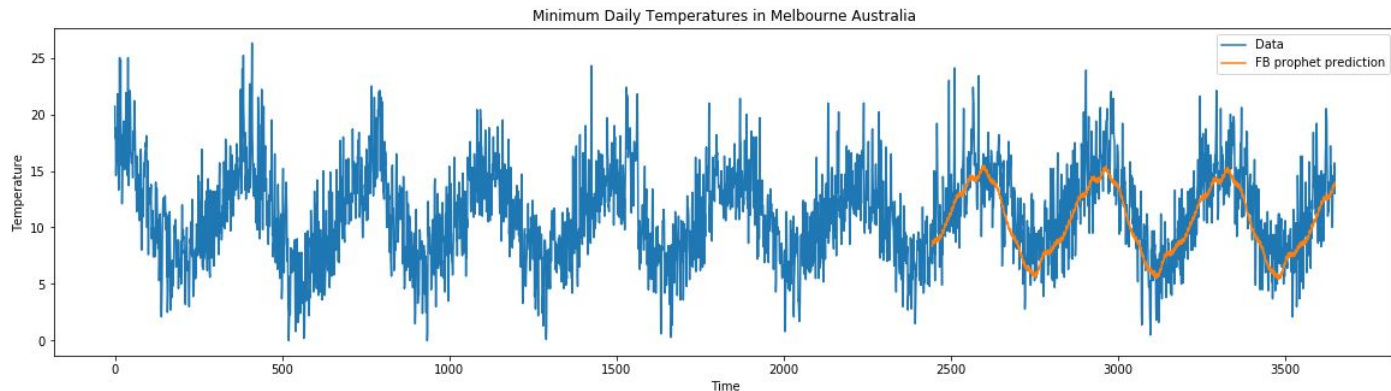
```
mean_squared_error(results.y, results.yhat)
```

```
8.239936336311873
```

Código: Prophet

Mostramos los resultados

```
[ ] index_start = len(train)
    index_end = len(train) + len(test)
    plt.figure(figsize = [20,5])
    plt.plot(data.Temp, label="Data")
    plt.plot(range(index_start, index_end), results.yhat.values, label="FB prophet prediction")
    plt.title("Minimum Daily Temperatures in Melbourne Australia")
    plt.xlabel("Time")
    plt.ylabel("Temperature")
    plt.legend()
    plt.show()
```

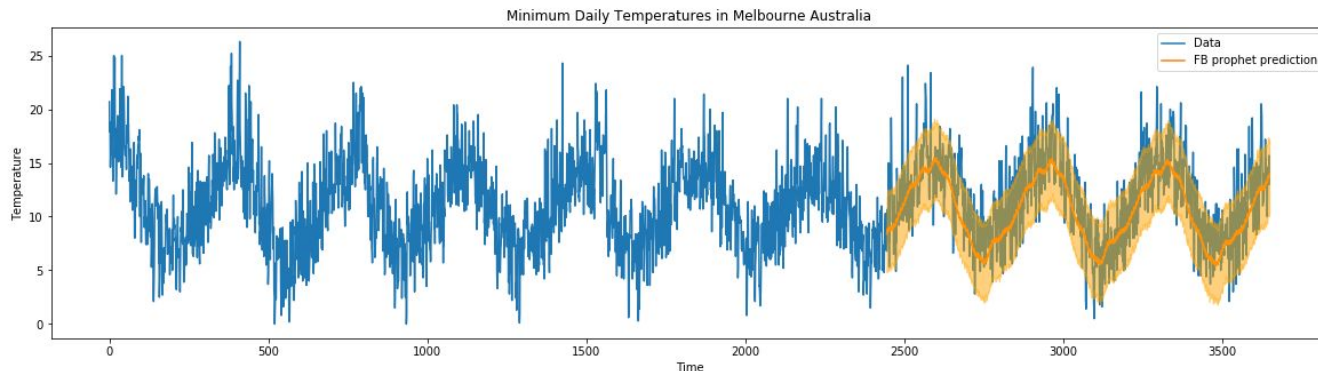


Código: Prophet

Mostramos los resultados con intervalos de error

```
plt.figure(figsize = [20,5])
plt.plot(data.Temp, label="Data", zorder = 1)
plt.plot(range(index_start, index_end), results.yhat.values, label="FB prophet prediction", zorder = 2)
plt.fill_between(range(len(train), len(train) + len(test)),
                 predictions.yhat_lower.values[index_start:index_end],
                 predictions.yhat_upper.values[index_start:index_end],
                 alpha=0.5, color = "orange", zorder = 3)

plt.title("Minimum Daily Temperatures in Melbourne Australia")
plt.xlabel("Time")
plt.ylabel("Temperature")
plt.legend()
plt.show()
```



¿Preguntas?

Federico Albanese

(falbanese@dc.uba.ar)

 @f__albanese
